

Laboratorio di Sistemi Operativi

Appunti del corso di Alessandra Rossi

A.A. 2023-2024

Indice

1	Introduzione	4
1.1	Unix	4
1.2	Filesystem	4
2	Utilizzo della shell	6
2.1	Shell	6
2.2	Variabili predefinite	6
2.3	Sintassi dei comandi	6
2.4	Variabili	6
2.5	File standard	7
2.6	Pipe (tubo)	8
2.7	Command substitution	8
2.8	Metacaratteri	8
2.9	Espressioni regolari di base	9
2.10	Processi	10
2.11	Word splitting	11
3	Linguaggio	12
3.1	Script	12
3.2	Linguaggio bash	13
3.3	Il comando <i>if</i>	13
3.4	Sostituzione	14
3.5	Il ciclo <i>while</i>	15
3.6	Il ciclo <i>for</i>	15
3.7	Case	16
4	Comandi	17
(*) : Richiede i permessi di superuser.		
4.1	echo - display a line of text	17
4.2	ls - list directory contents	17
4.3	cd - change directory	17
4.4	pwd - print working directory	18
4.5	cat - concatenate files and print on the standard output	18
4.6	man - an interface to the system reference manuals	18
4.7	chmod - change file mode bits	18
4.8	chown - change file owner	19
4.9	chgrp - change group ownership	19
4.10	mkdir - make directories	19
4.11	rmdir - remove empty directories	20
4.12	ln - make links between files	20
4.13	mv - move (rename) files	21
4.14	cp - copy files and directories	21
4.15	rm - remove files or directories	22
4.16	let	22
4.17	mount - mount a filesystem	22
4.18	grep - print lines that match patterns	23
4.19	ps - report a snapshot of the current processes	24
4.20	kill - send a signal to a process	25

4.21	wc - print newline, word, and byte counts for each file	25
4.22	sort - sort lines of text files	25
4.23	head/tail - output the first/last part of files	26
4.24	sed - stream editor for filtering and transforming text	26
4.25	awk - pattern scanning and processing language	29
4.26	(*) fdisk - manipulate disk partition table	31
4.27	ssh — OpenSSH remote login client	31
4.28	scp - OpenSSH secure file copy	32
4.29	(*) docker	32
4.30	netstat	34
5	Dischi e partizioni	35
5.1	Partizioni	35
5.2	Linux	37
6	Virtualizzazione	38
6.1	Esempi	38
6.2	Tipologie di virtualizzazione	39
6.3	Virtual Machine Monitor	39
6.4	VMM di sistema vs. VMM ospitati	40
6.5	Docker	41
6.6	Docker compose	42
6.7	Esempio di docker-compose	44
6.8	Ambiente di sviluppo ADE	46
6.9	VAGRANT	47
7	Linguaggio C	48
7.1	Sintassi	48
7.2	File	48
7.3	Operazioni sui file	49
7.4	Implementazione nel kernel	50
7.5	Condivisione dei file	51
7.6	Ottenere info su File	52
8	Processi	57
8.1	Funzione main	57
8.2	Processi	58
8.3	Identificazione dei processi	59
8.4	Creazione di processi	59
8.5	Terminazione di un processo	62
8.6	Segnali	64
9	Pipe	67
9.1	Interprocess Communication (IPC)	67
9.2	Pipe	67
9.3	La funzione Pipe	67
9.4	Pipe tra due programmi	69
9.5	popen e pclose	70
9.6	Pipe con nome (o FIFO)	71
10	Thread	73
10.1	Utilizzo dei thread	73
10.2	Esempio di utilizzo dei thread	75
10.3	Condivisione della memoria	75
10.4	Segnali e fork	76
10.5	Attributi di un thread	77
11	Concorrenza	79
11.1	Introduzione	79
11.2	Mutex Posix	80
11.3	Condition Variable	82
11.4	Thread specific data (TSD)	83

12 Socket	85
12.1 Introduzione	85
12.2 Socket API	86
12.3 Risoluzione dei nomi	90
13 Esercizi svolti	92
13.1 Variabili di Shell	92
13.2 grep	92
13.3 Processi	92
13.4 Concatenazione e sostituzione comandi	93
13.5 Word counter	93
13.6 Sort	93
13.7 Head e Tail	93
13.8 IFS	94
13.9 Script	94
13.10Linguaggio bash	95
13.11Il comando sed	95
13.12Il comando awk	95
13.13Input e Output	96
13.14Processi	97
13.15Pipe	98
13.16Thread	99

Introduzione

1.1 Unix

Unix è un sistema operativo con caratteristiche

- Multi-utente
- Multi-processo
- Filesystem gerarchico con radice unica
- Basato su kernel

kernel

Il kernel è il nucleo del sistema operativo, deve essere adattato ad ogni hardware, ed è l'unico che esegue comandi in modalità privilegiata.

Il kernel si occupa di:

- **CPU:** Lo scheduler stabilisce ad intervalli fissi e sulla base di priorità, quale processo deve essere mandato in esecuzione.
- **Memoria:** Il kernel gestisce la memoria virtuale, che consente di assegnare a ciascun processo uno spazio di indirizzi virtuale che poi viene rimappato sulla memoria fisica (RAM o disco).
- **Periferiche:** Le periferiche vengono viste attraverso un'interfaccia astratta che permette di trattarle come fossero file.

Tutto il resto anche l'interazione con l'utente è ottenuto tramite programmi eseguiti dal kernel.

Le interfacce con cui i programmi accedono all'hardware vengono chiamate *system call*: un insieme di funzioni che un programma può chiamare, per le quali viene generata un'interruzione del processo, passando il controllo dal programma al kernel.

Il kernel Unix nasce fin dall'inizio come sistema multiutente. Ogni utente ha un nome (*username*), una password e un identificativo numerico (*UID o user id*).

Sono previsti meccanismi di permessi e protezioni per impedire che utenti diversi possano danneggiarsi a vicenda o danneggiare il sistema.

È presente un utente speciale privilegiato (*superuser*), il cui username è solitamente *root* ed il cui *UID* è 0, che è l'amministratore di sistema.

1.2 Filesystem

Esistono tre tipi principali di file in Unix:

- File ordinari
- Directory
- File speciali

Il sistema assegna biunivocamente a ciascun file un identificativo, numerico detto *i-number*.

I file ordinari sono sequenze di byte (byte streams) e possono contenere informazioni di qualsiasi genere (dati, programmi sorgente, programmi oggetto..).

Il sistema non impone alcuna struttura interna.

Per consentire all'utente di rintracciare facilmente i propri file, Unix permette di raggrupparli in cartelle organizzate in un'unica struttura gerarchica ad albero.

Tra gli attributi di un file troviamo:

- Tipo
- Posizione
- Dimensione
- Numero di links
- Proprietario
- Permessi
- Creazione
- Modifica
- Accesso

Si può identificare un file tramite il suo path assoluto che descrive il cammino dalla root-directory al file (comincia per "/").

`/home/user/dir/subdir/file.txt`

Se un path non comincia per "/" allora si intende un path relativo alla working directory (directory in cui sta operando la shell corrente).

Utilizzo della shell

2.1 Shell

La shell è un programma che interpreta il linguaggio a linea di comando attraverso il quale l'utente utilizza le risorse del sistema. Nei sistemi Unix qualunque operazione può essere eseguita da una sequenza di comandi shell. Permette la gestione di variabili e dispone di costrutti per il controllo del flusso delle operazioni.

2.2 Variabili predefinite

Esistono delle variabili predefinite (chiamate anche variabili di ambiente) i cui caratteri per convenzione sono tutti maiuscoli.

Tra le principali ci sono:

- **PATH** percorso di ricerca eseguibili
- **USER** nome utente
- **HOME** directory home dell'utente
- **HOSTNAME** nome dell'hostname
- ...

2.3 Sintassi dei comandi

comando [*argomento...*]

Gli argomenti possono essere:

- **opzioni** (o flag) che sono preceduti da un "-"
- **parametri**

Una volta interpretata la prima parola (il comando) sulla linea di comando, la shell ricerca nel file system un file con il nome uguale a tale prima parola. La ricerca avviene ordinatamente all'interno delle directory elencate nella variabile d'ambiente *PATH*.

L'ordine delle opzioni e dei parametri è, **in genere**, irrilevante.

Unix è **CASE SENSITIVE**.

2.4 Variabili

Le variabili assumono lo stesso significato che assumono normalmente in un linguaggio di programmazione non tipizzato.

Per la scrittura su una variabile o la sua definizione avviene tramite la sintassi:

nomeVariabile=valore

È importante che non ci siano spazi tra la variabile e il valore.

Per la lettura la sintassi viene utilizzata la seguente sintassi:

\$variabile o in alternativa *\${variabile}*

È possibile svuotare il contenuto di una variabile con il comando *unset*.

Esempio di utilizzo

[echo]

```
# user@terminal: var1=7
# user@terminal: var2=5
# user@terminal: echo var1 var2
7 5
```

Le operazioni aritmetiche tra le variabili vengono effettuate tramite il comando let.

2.5 File standard

Normalmente un programma (o comando) opera su più file. I file standard che esistono in Unix sono:

- Standard input: il file su cui normalmente il programma acquisisce i suoi input
- Standard output: il file su cui normalmente il programma produce i suoi output
- Standard error: il file su cui normalmente il programma invia i messaggi di errore

In un normale sistema domestico, lo standard input è impostato sulla tastiera e lo standard output è impostato sullo schermo.

La shell può variare queste associazioni di default reindirigendo i file standard su qualsiasi altro file nel sistema.

Redirizione standard output

comando > file
comando >> file

Se il file non esiste viene creato.

Se il file esiste viene riscritto utilizzando > o il nuovo output viene accodato utilizzando >>.

Esempio di utilizzo

[ls, cat]

```
# user@terminal: ls
dir1 dir2 file1 file2
# user@terminal: ls -a > listaFile.txt
# user@terminal: cat listaFile.txt
dir1 dir2 file1 file2
```

Redirizione standard input

comando argomento1 ... < file

Il file viene rediretto sullo standard input del comando

Esempio di utilizzo

[ls, cat]

```
# user@terminal:
```

Redirizione standard error

comando 2 > file
comando 2 >> file

Se il file non esiste viene creato.

Se il file esiste viene riscritto utilizzando 2 > o il nuovo output viene accodato utilizzando 2 >>.

Esempio di utilizzo

[ls, cat]

```
# user@terminal: ls
dir1 dir2 file1 file2
# user@terminal: cd dir3 2> errori.txt
# user@terminal: cat errori.txt
-bash: cd: dir3: No such file or directory
```

2.6 Pipe (tubo)

comando1 | *comando2*

Lo standard output del *comando1* funge da standard input del *comando2*.
È necessario rispettare un determinato ordine:

- Possono stare solo all’inizio i comandi che scrivono su stdout (*echo*, *ls*...)
- Possono stare anche al centro i comandi che leggono da stdin e scrivono su stdout (*wc*, *sort*, *uniq*, *grep*, *cat*, *head*, *tail*)
- Possono stare solo alla fine i comandi per la paginazione (*less*, *more*)

Esempio di utilizzo

[echo, cat]

```
# user@terminal: echo topolino >> lista.txt
# user@terminal: echo paperino >> lista.txt
# user@terminal: echo pippo >> lista.txt
# user@terminal: cat lista.txt
topolino
paperino
pippo
# user@terminal: cat lista.txt | sort
paperino
pippo
topolino
```

2.7 Command substitution

Il pattern *\$comando* viene sostituito con l’output del comando

Esempio di utilizzo

[cd, cat]

filepath.txt contiene il percorso della cartella di destinazione
user@terminal: cd \$(cat filepath.txt)

2.8 Metacaratteri

La shell riconosce alcuni caratteri speciali, chiamati metacaratteri, che possono comparire nei comandi.
Quando l’utente invia un comando, la shell lo scandisce alla ricerca di metacaratteri che processa in modo speciale.

Esempio

[ls]

Ottiene tutti i file che terminano con .txt

```
# user@terminal: ls *.txt
file1.txt file2.txt file3.txt
```

I seguenti metacaratteri, detti wildcard sono utilizzati per indicare insiemi di nomi in un pathname.

Metacarattere	Significato
*	Stringa di 0 o più caratteri
?	Singolo carattere
[]	Singolo carattere tra quelli elencati
{ }	Stringa tra quelle elencate

Quoting

Attraverso il quoting è possibile inibire l'effetto dei metacaratteri.

Ci sono tre meccanismi di quoting:

- Il metacarattere di **escape** \inibisce l'effetto del metacarattere che lo segue.
- Tutti i metacaratteri presenti in una stringa racchiusa tra **singoli apici** perdono l'effetto speciale.
- I metacaratteri perl'abbreviazione del pathname presenti in una stringa racchiusa tra **doppi apici** perdono l'effetto speciale (ma non tutti i metacaratteri della shell).

Metacaratteri di shell

Metacarattere	Significato
>	Ridirezione dell'output
>>	Ridirezione dell'input (append)
<	Ridirezione dell'input
<< delim	Ridirezione dell'input da linea di comando
*	Wildcard: stringa di 0 o più caratteri ad eccezione del punto
?	Wildcard: un singolo carattere ad eccezione del punto
[...]	Wildcard: un singolo carattere tra quelli elencati
{...}	Wildcard: le stringhe tra quelle elencate
	Pipe
;	Sequenza di comandi
	Sequenza condizionale (Esegue un comando se il precedente fallisce)
&&	Esecuzione condizionale (Esegue un comando se il precedente termina con successo)
#	Introduce un commento
\	Fa in modo che la shell non interpreti in modo speciale il carattere che segue
!	Ripetizione di comandi memorizzati nell'history list (CTRL+R per cercare tra i comandi)

2.9 Espressioni regolari di base

Le seguenti espressioni regolari fanno riferimento in particolare al comando grep, ma molte fanno ugualmente parte dello standard POSIX e della shell bash

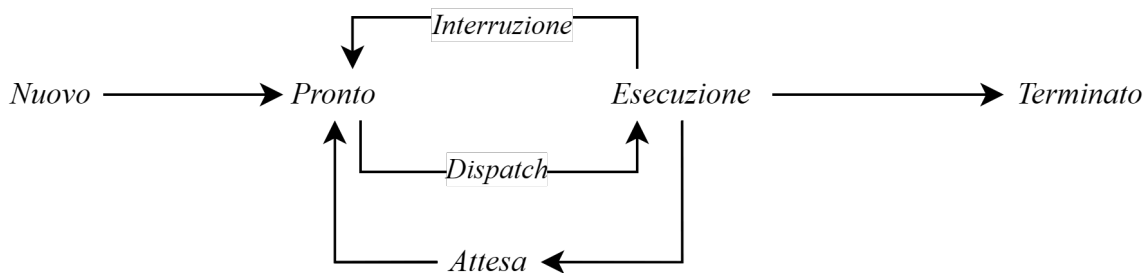
Espressione	Significato	Standard
.	Qualunque carattere	grep
exp*	Zero o più occorrenze di exp	grep
^exp	exp all'inizio del rigo	grep
exp\$	exp alla fine del rigo	grep
[a-z]	un carattere nell'intervallo specificato	bash/grep
[^a-z]	un carattere fuori dall'intervallo specificato	bash/grep
\<exp	exp all'inizio di una parola	grep
exp\>	exp alla fine di una parola	grep
exp{N}	exp compare N volte	grep
exp{N,M}	exp compare almeno N ed al più M volte	grep
[[:CLASS:]]	un carattere in CLASS	POSIX/grep
[[:alpha:]]	I caratteri alfabetici	POSIX/grep
[[:alnum:]]	I caratteri alfanumerici	POSIX/grep
[[:digit:]]	Le cifre	POSIX/grep
[[:upper:]]	I caratteri alfabetici maiuscoli	POSIX/grep
[[:lower:]]	I caratteri alfabetici minuscoli	POSIX/grep

2.10 Processi

I processi sono programmi in esecuzione. Lo stesso programma può corrispondere a diversi processi. Ogni processo può generare nuovi processi detti processi *figli*. Ogni processo ha:

- *Process Identification Number (PID)*
- *Parent Process Identification Number (PPID)*

Il processo `init` (la radice della gerarchia dei processi che si avvia all'avvio del sistema) ha `PID=1` e nessun `PPID`.



Ciclo di vita del processo

Il kernel gestisce una tabella dei processi che serve a tenere traccia del loro stato. In particolare sono registrati i seguenti valori:

- Il nome dell'eseguibile in funzione
- Gli eventuali argomenti passati all'eseguibile al momento dell'avvio attraverso la riga di comando
- Il numero di identificazione del processo
- Il numero di identificazione del processo che lo ha generato
- Il nome del dispositivo di comunicazione se il processo è controllato da un terminale
- Il numero di identificazione dell'utente
- Il numero di identificazione del gruppo

Attributi dei processi

Ad ogni processo sono associati due utenti

- **Real user:** utente che ha lanciato il processo.
- **Effective user:** utente che determina i diritti del processo.

Quando il processo apre un file, vale l'*effective user*. Di solito i due utenti coincidono se invece un file eseguibile ha il bit "*set user ID*" impostato.

Ogni processo ha anche due gruppi associati (*real group* ed *effective group*).

Un programma con "*set user ID*" impostato è *ping*:

```
# user@terminal: ls -l /bin/ping
-rwsr-xr-x 1 root root 30804 2023-10-16 19:32
```

Il terzo bit infatti è impostato su **s** invece che su **x**.

ping ha bisogno di partire con i permessi di amministratore.

Controllo dei processi

Normalmente la shell aspetta che ogni comando termini (comando in foreground), ma utilizzando la sintassi

comando&

è possibile eseguire un comando in background (Il comando può comunque scrivere sullo standard output).

I processi in background sono eseguiti in una sottoshell, in parallelo al processo padre, e non sono controllabili da tastiera.

Non bisogna confondere un job di shell con un processo: un job di shell rappresenta tutti i processi che vengono generati da un comando impartito tramite la shell stessa. Per esempio quando viene avviato uno script che avvia a sua volta diversi programmi.

Un job può essere sospeso con la combinazione di tasti *CTRL+Z* e può essere ripreso con *fg* se il job era in foreground o con *bg* se il job era in background.

2.11 Word splitting

L'ultima fase prima di eseguire un comando consiste nella suddivisione in parole.

La variabile **IFS** (*internal field separator*) definisce i separatori.

Di default, IFS="<space><tab><newline>"

Come effetto collaterale, il word splitting sostituisce i newline con spazi.

Esempio con IFS='\\n'

```
# user@terminal: var='$'Hello Nico\\nHello Nico!'
# user@terminal: IFS='\\n'
# user@terminal: echo -e $var
Hello Nico
Hello Nico!
```

Esempio con IFS=' '

```
# user@terminal: var='$'Hello Nico\\nHello Nico!'
# user@terminal: IFS=' '
# user@terminal: echo -e $var
Hello
Nico
Hello
Nico!
```

Esempio con array

```
# user@terminal: text="1|tom|1982"
# user@terminal: IFS='|'
# user@terminal: array=($text)
# user@terminal: echo ${array[0]} ${array[1]} ${array[2]}
1 tom 1982
```

¹Non mi funziona davvero...

Linguaggio

3.1 Script

Uno script di shell BASH è un file di testo che inizia con:

`#!/bin/bash`

e che ha il permesso di esecuzione. Il resto del file contiene comandi di shell.

Può essere eseguito invocando `./nomeScript` o in alternativa `bash nomeScript`.

Non c'è differenza tra quello che si può scrivere al prompt e quello che si può scrivere in uno script.

L'esecuzione di un comando `cd` o la definizione di una variabile avranno effetto solo all'interno dello script e non cambieranno la working directory dell'utente e le sue variabili.

I primi due caratteri dicono a bash che il file è uno script, il resto dice a bash qual è l'interprete per questo script.

Risultato: viene invocato l'interprete passandogli come argomento il nome dello script.

Variabili definite

- **`$0`** Il nome dello script stesso (`argv[0]`)
- **`$1...$i`** i parametri da riga di comando (`argv[i]`)
- **`$#`** numero di parametri ricevuti
- **`$*`** tutti i parametri in una stringa singola
- **`$@`** tutti i parametri in stringhe separate
- **`$!`** process ID (PID) del processo corrente
- **`$?`** exit status dell'ultimo comando eseguito

Exit

Ogni comando restituisce un intero detto *exit status* al chiamante.

Di norma:

- **`0`** terminazione regolare
- **Diverso da zero** terminazione irregolare

La variabile di shell `$?` contiene l'exit status dell'ultimo comando eseguito.

Input da terminale

Attraverso il comando `read` è possibile permettere all'utente di inserire valori a runtime di uno script.

Esempio

```
#!/bin/bash
echo "Inserisci un numero: "
read n
echo "Il numero che hai inserito è: $n"
```

3.2 Linguaggio bash

Operatori sui comandi

- `cmd1; cmd2`
Esegue `cmd1` seguito da `cmd2`.
- `cmd1 && cmd2`
Esegue `cmd1` e, se `cmd1` è terminato con successo (codice errore 0), esegue `cmd2`.
- `cmd1 || cmd2`
Esegue `cmd1` e, se `cmd1` NON è terminato con successo (codice errore diverso da 0), esegue `cmd2`.

In tutti e tre i casi l'exit status è quello dell'ultimo comando eseguito.

3.3 Il comando *if*

Come test si usa l'exit stato del comando
Per mettere *if* e *then* stessa linea, utilizzare il punto e virgola.

```
if comando
then
    lista comandi
[elif comando
    lista comandi ]
[else comando
    lista comandi ]
fi
```

Espressioni condizionali

Il comando "test exp" valuta exp come espressione condizionale (cioè termina con 0 se exp è vera).
"test exp" si può abbreviare con "[exp]" (spazi obbligatori).

Operatori su stringhe	
STR1 = STR2	the strings are equal
STR1 != STR2	the strings are not equal
-z STR	the length of STRING is zero
-n STR	the length of STRING is nonzero
Operatori su interi	
INT1 -eq INT2	INT1 is equal to INT2
INT1 -ge INT2	INT1 is greater than or equal to INT2
INT1 -gt INT2	INT1 is greater than INT2
INT1 -le INT2	INT1 is less than or equal to INT2
INT1 -lt INT2	INT1 is less than INT2
INT1 -ne INT2	INT1 is not equal to INT2
Operatori unari su file	
-e FILE	FILE exists

-f FILE	FILE exists and is a regular file
-r FILE	FILE exists and read permission is granted
-w FILE	FILE exists and write permission is granted
-x FILE	FILE exists and execute (or search) permission is granted
Operatori booleani	
EXP	EXP is true
! EXP	EXP is false
EXP1 -a EXP2	both EXP1 and EXP2 are true
EXP1 -o EXP	either EXP1 or EXP2 is true

Esempio

Verifica se abbiamo passato almeno un argomento:

```
if [ -z "$1" ]
then
    echo "Questo script richiede un argomento."
    exit 1
fi
```

Verifica che ci siano 4 argomenti altrimenti verifica che il file \$1 esiste:

```
if [ $# -lt 4 ]
then
    echo "Questo script richiede 4 argomenti."
    exit 1
elif [ ! -e $# -lt 4 ]
    echo "Il file $1 non esiste."
    exit 1
fi
```

3.4 Sostituzione

$\$(exp)$ valuta *exp* come espressione aritmetica.

$\$(exp)$ viene sostituito dalla shell con il valore di *exp*.

Funziona solo con aritmetica su numeri interi.

Operatori ammessi:

- Aritmetici: +, -, /, *, %
- Elevamento a potenza: **
- Bit-a-bit: <<, >>, &, |, ~ ¹
- Booleani: <, <=, ==, !=, >, >=, ||, &&, !
- Unari: var++, var--

Esempio

Se la variabile **var** vale 7

$\$(($+1))$ viene sostituito con 8.

$\$(a+1)$ viene sostituito con 8.

$\$(a++)$ viene sostituito con 7 ma viene incrementata a.

$\$(a*3>8)$ viene sostituito con 1 perché è vero (0 per falso).

¹L'operatore bit-a-bit controlla tutti i bit, il booleano si ferma appena può dedurre la soluzione

3.5 Il ciclo *while*

Ripete una sequenza di comandi fintantoché un comando viene eseguito con successo (ovvero la condizione è **vera**).

```
while comando
do
    sequenza
    comandi
done
```

Esempio

Incrementa di 10 valori:

```
i=0
while [ $1 -lt 10 ]
do
    i=$((i+1))
done
```

Il ciclo *until*

A differenza del ciclo *while*, il ciclo *until* si ripete fintantoché la condizione è **falsa**.

3.6 Il ciclo *for*

Ripete una sequenza di comandi iterando una variabile in una lista di valori.

```
for var in lista valori
do
    sequenza
    comandi
done
```

Esempio

Iterando la variabile *var*

```
for var in 1 2 3 var assume i valori 1, 2 e 3
for var in $(ls) var assume i valori dei file nella working directory
for var in "uno" "due" "tre" var assume i valori delle stringhe "uno", "due" e "tre"
for var in $@ var assume i valori passati per linea di comando ($* è un'unica stringa)
for var in *.txt var assume i valori dei file di testo nella working directory
```

3.6.1 Differenza tra \$* e \$@

Il seguente script

```
#!/bin/bash
for x in "$*"
do
    echo "x: $x"
done
x=$1
for x in "$@"
do
    echo "x: $x"
done
```


stampa come output:

```
x:  1 2 3
```

```
x:  1
```

```
x:  2
```

```
x:  3
```

Il primo ciclo itera una sola volta su una stringa che contiene tutti i parametri passati.

Il secondo ciclo itera una volta per ogni parametro passato.

3.7 Case

Permette di eseguire determinate operazione confrontando delle stringhe.

Se la variabile da verificare è *string*

```
case string in
    stringa1) lista di comandi 1 ;;
    stringa2) lista di comandi 2 ;;
    ...)  lista di comandi ...  ;;
    *) lista di comandi di default ;;
esac
```

Comandi

4.1 echo - display a line of text

`echo [OPZIONI] [TESTO]`

Permette di stampare un messaggio sullo standard output.
Non sono mai state utilizzate opzioni

Esempio di utilizzo

```
# user@terminal: echo hello world
hello world
```

4.2 ls - list directory contents

`ls [OPZIONI] [CARTELLA]`

Permette di listare i file in una cartella.
CARTELLA è la cartella in cui effettuare il listing e di default è impostata sulla working directory.

Opzioni utilizzate

- a** (*all*) lista tutti i file, non ignorando i file nascosti che iniziano con `.`
- l** (*long list format*) lista i file con tutte le informazioni
- r** (*reverse*) inverte l'ordine della lista
- R** (*recursive*) lista anche il contenuto delle sottocartelle
- s** (*size*) mostra la dimensione dei file
- sort=WORD** ordina in base alla *WORD* che può essere size, time, extension...
- i** mostra l'i-number.

Esempio di utilizzo

```
# user@terminal: ls -l
drwxr-xr-x 2 proprietario gruppo 4096 Sep 29 13:25 nomecartella
-rw-r--r-- 1 proprietario gruppo 1298 Sep 26 14:28 nomefile
```

4.3 cd - change directory

`cd [CARTELLA]`

Permette di cambiare la working directory partendo da quella attuale
CARTELLA può essere specificato o in maniera assoluta (inserendo all'inizio del comando) o in maniera relativa rispetto alla working directory attuale.
"`.`" permette di riferirsi alla cartella attuale, "`..`" permette di riferirsi alla cartella padre.

4.4 pwd - print working directory

pwd [*OPZIONI*]

Permette di stampare il percorso assoluto della working directory.
Non sono mai state utilizzate opzioni per questo comando.

Esempio di utilizzo

```
# user@terminal: pwd
/home/user/dir/subdir
```

4.5 cat - concatenate files and print on the standard output

cat [*OPZIONI*] [*FILES*]

Permette di stampare il contenuto di più *FILES* sullo standard output.
Non sono mai state utilizzate opzioni per questo comando.

Esempio di utilizzo

```
# user@terminal: cat textfile.txt
contenuto del file di testo
```

4.6 man - an interface to the system reference manuals

man [*OPZIONI*] [*COMANDO*]

Permette di avere una descrizione dettagliata sull'utilizzo del *COMANDO* e sulle sue opzioni.
Non sono mai state utilizzate opzioni per questo comando.

Esempio di utilizzo

```
# user@terminal: man ls
NAME
ls - list directory contents
```

```
SYNOPSIS
ls [OPTION]... [FILE]...
```

```
DESCRIPTION
List information about the FILES (the current directory by default). Sort entries alphabetically if none of -cftuvSUX
nor --sort is specified.
```

4.7 chmod - change file mode bits

chmod [*OPZIONI*] [*UTENTE*][*CAMBIAMENTO*][*PERMESSI*] [*FILE*]

Permette di cambiare i permessi di lettura, scrittura, esecuzione di un file per il proprietario, gruppo, altri utenti, tutti.

Tra l'utente, il cambiamento e i permessi non ci devono essere spazi.

Opzioni utilizzate

-R (*recursive*) cambia i permessi a tutti file e le sottocartelle

Utente

- u** (*user*) permette di cambiare i permessi al proprietario
- g** (*group*) permette di cambiare i permessi al gruppo
- o** (*other*) permette di cambiare i permessi agli esterni al gruppo
- a** (*user*) permette di cambiare i permessi a tutte e tre le categorie

Cambiamento

- +** (*add*) permette di aggiungere permessi
- (*remove*) permette di rimuovere permessi
- =** (*assign*) permette di sovrascrive i permessi

Permessi

- r** (*read*) permessi di lettura
- w** (*write*) permessi di scrittura
- x** (*execute*) permette di sovrascrive i permessi

Esempio di utilizzo

```
# user@terminal: chmod u+w fileDaScrivere.txt
```

4.8 chown - change file owner

chown [*OPZIONI*] [*UTENTE*] [*FILES*]

Permette di impostare la proprietà del file su un nuovo utente.
Solo il proprietario del file ed il superuser possono modificare queste impostazioni.
Non sono mai state utilizzate opzioni per questo comando.

Esempio di utilizzo

```
# user@terminal: chown newUser file.txt
```

4.9 chgrp - change group ownership

chgrp [*OPZIONI*] [*GRUPPO*] [*FILES*]

Permette di impostare la proprietà del file su un nuovo gruppo.
Solo il proprietario del file ed il superuser possono modificare queste impostazioni.
Non sono mai state utilizzate opzioni per questo comando.

Esempio di utilizzo

```
# user@terminal: chgrp newGrp file.txt
```

4.10 mkdir - make directories

mkdir [*OPZIONI*] [*PERCORSO/NOME*]

Permette di creare una nuova cartella.
PERCORSO/ è di default impostato su *./* per creare una cartella nella working directory. Non sono mai state utilizzate opzioni per questo comando.

Esempio di utilizzo

```
# user@terminal: ls

# user@terminal: mkdir dir1 dir2
# user@terminal: ls
dir1 dir2
```

4.11 rmdir - remove empty directories

rmdir [*OPZIONI*] [*PERCORSO/NOME*]

Permette di creare una nuova cartella.

PERCORSO/ è di default impostato su *./* per rimuovere una cartella nella working directory.

È necessario che la cartella sia vuota per essere eliminata. Non sono mai state utilizzate opzioni per questo comando.

Esempio di utilizzo

```
# user@terminal: ls
dir1 dir2
# user@terminal: rm dir1
# user@terminal: ls
dir2
```

4.12 ln - make links between files

ln [*OPZIONI*] [*FILE1*] [*DESTINAZIONE/NOME*]

Permette di creare un collegamento al *FILE1* nella destinazione.

Se cancellato il file originale, il collegamento continua ad esistere. (hardlink)

Qualunque modifica apportata al collegamento si ripercuote sul file originale e viceversa.

Opzioni utilizzate

-s (*symbolic*) permette di creare link simbolici (Se cancellato il file originale il collegamento diventa inutilizzabile).

Esempio di utilizzo hardlink

```
# user@terminal: cat file1.txt
testo del file1
# user@terminal: ln file1.txt linkfile1
# user@terminal: cat linkfile1
testo del file1
# user@terminal: rm file1.txt
# user@terminal: cat linkfile1
testo del file1
```

Esempio di utilizzo softlink

```
# user@terminal: cat file1.txt
testo del file1
# user@terminal: ln -s file1.txt linkfile1
# user@terminal: cat linkfile1
testo del file1
# user@terminal: rm file1.txt
```

```
# user@terminal: cat linkfile1
cat: linkfile1: No such file or directory
```

4.13 mv - move (rename) files

mv [*OPZIONI*] [*FILES*] [*DESTINAZIONE*]

Permette di spostare o rinominare un file.

Se *DESTINAZIONE* è una directory i file vengono spostati nella directory di destinazione.

Se *DESTINAZIONE* non esiste viene creato un file con quel nome.

Non sono mai state utilizzate opzioni per questo comando.

Esempio di utilizzo per spostare

```
# user@terminal: ls
dir1 dir2
# user@terminal: ls dir1
file1.txt
# user@terminal: ls dir2

# user@terminal: mv dir1/file1.txt dir2
# user@terminal: ls dir2
file1.txt
```

Esempio di utilizzo per rinominare

```
# user@terminal: ls
file1.txt
# user@terminal: mv file1.txt file2.txt
# user@terminal: ls
file2.txt
```

4.14 cp - copy files and directories

cp [*OPZIONI*] [*SORGENTE*] [*DESTINAZIONE*]

Permette di copiare un file o una cartella.

Se *DESTINAZIONE* è una directory i file vengono copiati nella directory di destinazione (mantenendo il nome originale).

Se *DESTINAZIONE* non esiste viene creato un file con quel nome.

A differenza di link, le copie non sono in alcun modo collegate tra loro.

Non sono mai state utilizzate opzioni per questo comando.

Esempio di utilizzo

```
# user@terminal: ls
file1.txt
# user@terminal: cat file1.txt
testo del file1.txt
# user@terminal: cp file1.txt file2.txt
# user@terminal: cat file2.txt
testo del file1.txt
```

4.15 rm - remove files or directories

mv [*OPZIONI*] [*FILES*]

Permette di un file o una cartella.

Se il file da cancellare è un cartella, viene visualizzato un messaggio di errore a meno di utilizzare la corretta opzione.

Opzioni utilizzate

-r (*recursive*) permette di eliminare le cartelle e tutti i file contenuti.

Esempio di utilizzo

```
# user@terminal: ls
file1.txt
# user@terminal: cat file1.txt
testo del file1.txt
# user@terminal: cp file1.txt file2.txt
# user@terminal: cat file2.txt
testo del file1.txt
```

4.16 let

let [*ESPRESSIONE*]

Permette di eseguire espressioni aritmetiche sulle variabili

Esempio di utilizzo

```
# user@terminal: a=3
# user@terminal: b=7
# user@terminal: echo $a+$b
3+7
# user@terminal: let c=a+b
# user@terminal: echo $c
10
```

4.17 mount - mount a filesystem

mount -o [*OPZIONI*] -t [*TIPO*] [*DISPOSITIVO*] [*DESTINAZIONE*]

Permette di montare un filesystem in una cartella.

DISPOSITIVO è il file a cui fa riferimento il dispositivo.

DESTINAZIONE è la cartella in cui si vuole montare il filesystem per utilizzare il dispositivo.

unmount - unmount a filesystem

unmount [*DISPOSITIVO o DESTINAZIONE*]

Utilizzato per smontare tutti i filesystem relativi a quel dispositivo o montati in quella cartella.

Opzioni

rw (*read and write*) monta il filesystem in lettura e scrittura.(default)

ro (*read only*) monta il filesystem in sola lettura.

exec (*execution*) monta il filesystem permettendo l'esecuzione dei file.

noexec (*no execution*) monta il filesystem negando l'esecuzione dei file.

remount rimonta un filesystem esistente (Utilizzato per modificarne le opzioni).

Tipo

ext2	Linux
reiserfs	Linux
ext3	Linux
vfat	Windows
msdos	MS-DOS
ntfs	Windows
iso9660	CD-ROM
hpfs	OS/2
hfs	Macintosh

Esempio di utilizzo per montare

Filesystem presente sulla partizione /dev/hda1 in /mnt/windows

```
# user@terminal: mount /dev/hda1 /mnt/windows
```

Esempio di utilizzo per smontare

Filesystem presente sulla partizione /dev/hda1

```
# user@terminal: umount /dev/hda1
```

4.18 grep - print lines that match patterns

grep [*OPZIONI*] [*PATTERN*] [*FILE*]

Permette di stampare le righe del file che corrispondono al pattern.

Il pattern è un'espressione regolare (Singola o complessa).

Se *FILE* non è specificato, legge da standard input. Questo consente la concatenazione in *pipe*

Opzioni utilizzate

-v (*invert*) stampa le righe che non corrispondono al pattern.

-c (*count*) stampa solo il numero di occorrenze.

-i (*case-Insensitive*) non differisce tra maiuscole e minuscole (di default è case-sensitive).

-n (*number*) permette di ottenere il numero della riga in cui si è trovata l'occorrenza.

Esempio di utilizzo

Stampa i file che sono stati modificati l'ultima volta ad ottobre

```
# user@terminal: ls -l | grep Oct
drwxr-xr-x 5 user group 4096 Oct 7 13:14 dir1
-rw-r--r-- 1 user group 1024 Oct 8 13:51 file1.txt
```

Esempio di utilizzo *invert*

Stampa i file che non sono stati modificati l'ultima volta ad ottobre

```
# user@terminal: ls -l | grep -v Oct
drwxr-xr-x 5 user group 4096 Nov 7 13:14 dir2
-rw-r--r-- 1 user group 1024 Nov 8 13:51 file2.txt
```


Esempio di utilizzo *count*

Stampa il numero dei file che sono stati modificati l'ultima volta ad ottobre

```
# user@terminal: ls -l | grep -c Oct
2
```

Esempio di utilizzo con espressioni

[Espressioni regolari]

Stampa le cartelle presenti nella working directory

```
# user@terminal: ls -l | grep ^d
drwxr-xr-x 5 user group 4096 Oct 7 13:14 dir1
drwxr-xr-x 5 user group 4096 Nov 7 13:14 dir2
```

Esempio di utilizzo con espressioni

[Espressioni regolari]

Stampa tutti i file che non sono cartelle e che hanno il permesso di esecuzione per l'utente

```
# user@terminal: ls -l | grep -v ^d | grep ^...x
-rwxr-xr-x 5 user group 4096 Oct 7 13:14 file1
```

Esempio di utilizzo con espressioni

[Espressioni regolari]

Stampa tutti i file che hanno un trattino al secondo posto nel nome. Sono necessari i caratteri di escape per permettere alla shell di ignorare i metacaratteri dell'espressione regolare.

```
# user@terminal: ls | grep '\<.-'
f-file1
```

4.19 ps - report a snapshot of the current processes

ps [*SELEZIONE*] [*FORMATO*]

Fornisce i PID, terminale, ora di esecuzione e comando relativo ai processi dell'utente associati al terminale corrente

Selezione

- u utente** (*user*) processi lanciati dall'utente specificato
- a** (*all*) tutti i processi

Formato

- f** (*full*) aggiunge UID, PPID e gli argomenti
- F** (*Full*) aggiunge anche altre informazioni
- o campo** visualizza solo i campi specificati

Esempio di utilizzo

Stampa i file che sono stati modificati l'ultima volta ad ottobre

```
# user@terminal: ps -u $USER
```

4.20 kill - send a signal to a process

kill [*OPZIONI*] [*PID*]

Permette di terminare un processo.

Per terminare un processo in esecuzione nella shell si ottiene lo stesso effetto premendo *CTRL+C*.

PID è il PID del programma da terminare.

Opzioni utilizzate

-s (*signal*) invia un segnale specifico al processo

-l (*list*) elenca i segnali disponibili

Esempio di utilizzo

Termina il programma con PID=12345

```
# user@terminal: kill 12345
```

Esempio di utilizzo con segnale

Termina il programma con PID=12345 inviando il segnale SIGKILL

```
# user@terminal: kill -s SIGKILL 12345
```

4.21 wc - print newline, word, and byte counts for each file

wc [*OPZIONI*] [*FILE*]

Fornisce il numero dei codici di interruzione di riga, delle parole o dei caratteri contenuti nel *FILE*.

Opzioni utilizzate

-c (*character*) conta il numero dei caratteri

-w (*word*) conta il numero delle parole

-l (*line*) conta il numero delle linee

Senza alcuna opzione restituisce tutte e tre le informazioni

Esempio di utilizzo

```
# user@terminal: cat testo.txt
```

Questo

è un

testo di prova

```
# user@terminal: wc testo.txt
```

```
3 6 28 testo.txt
```

4.22 sort - sort lines of text files

sort [*OPZIONI*] [*FILE*]

Permette di riordinare o fondere insieme il contenuto dei file passati come parametri, oppure di riordinare le linee passategli in input.

In assenza di opzioni viene considerato l'ordine alfabetico.

Opzioni utilizzate

- f Ignora le differenze tra maiuscole e minuscole
- n (*numerical*) considera numerica anzichè testuale la chiave di ordinamento
- r (*reverse*) ordina in senso inverso/decrescente
- t *s* utilizza *s* come separatore di campo
- o *fileout* (*output*) invia l'output a *fileout* anzichè sull'output standard
- k *s1, s2* (*key*) utilizza i campi da *s1* a *s2-1* come chiavi di ordinamento

Esempio di utilizzo

```
# user@terminal: ls
1file file1 file2 h-file
# user@terminal: ls | sort -r
h-file
file2
file1
1file
```

4.23 head/tail - output the first/last part of files

```
head [-n] [FILE]
tail [-n] [FILE]
```

Head restituisce le prime *n* linee del *FILE*

Tail restituisce le ultime *n* linee del *FILE*

Esempio di utilizzo

```
# user@terminal: head -4 testo.txt
O in alternativa # user@terminal: head -n 4 testo.txt
```

4.24 sed - stream editor for filtering and transforming text

[Espressioni regolari]

```
sed [OPZIONI] [COMANDI] [INDIRIZZO] [INPUT FILE]
```

sed viene definito un edito di testo non interattivo.

Fa uso delle espressioni regolare per modificare l'input fornito (Di default sullo standard input).

Restituisce di default l'output sullo standard output (Ma può essere rediretto) modificato dagli eventuali *COMANDI*.

sed non modifica l'input, ovvero se gli viene passato un file, questo non verrà modificato.

Consente di effettuare le seguenti operazioni:

- sostituzioni
- cancellazioni di linee
- aggiunta di linee
- rimpiazzamento di linee

L'*INDIRIZZO* serve a delimitare l'azione dei *COMANDI* e può essere espresso sotto la forma di:

- Un numero (Che conta le linee di input nel file di input)
- Un carattere '\$' per l'ultima linea di input
- Un address di contesto (Espressione regolare preceduta o seguita da un delimitatore)

Esempi di indirizzi

Notazione	Effetto
8d	Cancella l'ottava riga dell'input.
/^\$/d	Cancella tutte le righe vuote.
/^\$/d	Cancella dall'inizio dell'input fino alla prima riga vuota compresa.
/Jones(d	Visualizza solo le righe in cui è presente "Jones".
s/Windows/Linux/	Sostituisce con "Linux" la prima occorrenza di "Windows" trovata in ogni riga dell'input.
s/BSOD/stabilità/g	Sostituisce con "stabilità" tutte le occorrenze di "BSOD" trovate in ogni riga dell'input.
s/*\$//	Cancella tutti gli spazi che si trovano alla fine di ogni riga.
s/00*/0/g	Riduce ogni sequenza di zeri ad un unico zero.
/GUI/d	Cancella tutte le righe in cui è presente "GUI".
s/GUI//g	Cancella tutte le occorrenze di "GUI", lasciando inalterata la parte restante di ciascuna riga.

Opzioni utilizzate

-e permette di concatenare comandi.

-n permette di non duplicare l'output rispamando l'input originale.

Comandi utilizzati

Operatore	Nome	Effetto
[indirizzo]/p	print	Visualizza l'indirizzo specificato
[indirizzo]/d	delete	Cancella l'indirizzo specificato
[indirizzo]s/modello1/modello2/	substitute	Sostituisce in ogni riga la prima occorrenza della stringa modello1 con la stringa modello2 nell'indirizzo specificato
[indirizzo]/y/modello1/modello2	transform	Sostituisce tutti i caratteri della stringa modello1 con i corrispondenti caratteri della stringa modello2 nell'indirizzo specificato
comando/g	global	Applica su tutte le occorrenze il comando specificato

Il funzionamento di *sed* è quello di una macchina a registri:

1. Copia ciclicamente una linea di input in un pattern space (non modifica l'input)
2. Applica tutti i comandi con address selezioni dal pattern space
3. Copia il pattern space nello standard output, aggiungendo newline
4. Cancella il patterna space

Esempio di utilizzo

Dato il seguente testo contenuto nel file testo.txt:

```
BEGIN
Line 1 of input
Line 2 of input
Line 3 of input
END
Line 4 of input
```

Line 5 of input

Seleziona le righe comprese tra "BEGIN" e "END"

```
# user@terminal: sed -n '/^BEGIN$/,/^END$/p' testo.txt
```

BEGIN

Line 1 of input

Line 2 of input

Line 3 of input

END

O in alternativa

```
# user@terminal: sed -n '1,5p' testo.txt
```

BEGIN

Line 1 of input

Line 2 of input

Line 3 of input

END

Cancella la prima riga

```
# user@terminal: sed '1d' testo.txt
```

Line 1 of input

Line 2 of input

Line 3 of input

END

Line 4 of input

Line 5 of input

Stampa tutte le righe che contengono "input" (Comportamento in assenza del -n)

```
# user@terminal: sed '/input/p' testo.txt
```

BEGIN

Line 1 of input

Line 1 of input

Line 2 of input

Line 2 of input

Line 3 of input

Line 3 of input

END

Line 4 of input

Line 4 of input

Line 5 of input

Line 5 of input

Stampa tutte le righe che contengono "input" (Comportamento in presenza del -n)

```
# user@terminal: sed -n '/input/p' testo.txt
```

Line 1 of input

Line 2 of input

Line 3 of input

Line 4 of input

Line 5 of input

Cancella tutte le parole "input" e "Line"

```
# user@terminal: sed -e 's/input//g' -e 's/Line//g' testo.txt
```

BEGIN

1 of

2 of

3 of

END

4 of

5 of

4.25 awk - pattern scanning and processing language

awk [*OPZIONI*] [*PROGRAMMA*] [*FILE*]

Funzione awk cerca su file linee o altre unità di testo che contengono pattern.

Quando una linea corrisponde ad un pattern, azioni speciali vengono eseguite sulla linea.

In awk i programmi sono "data-driven": descrivi cosa cerchi, poi esegui.

Opzioni utilizzate

-f (*file*) selezionare un file script

-F Cambiare i separatori di campo/record

Il *PROGRAMMA* e' definito da un insieme di regole:

pattern { action }

Dove *pattern* è uno tra

- **BEGIN** prima di processare l'input
- **END** dopo aver processato l'input
- **boolexp** fa match se è vera (es. \$1 == "ciao")
- **/regex/** fa match se la regex fa match (es. /^July/)

[Reg.Expr.]

E *action* è uno tra

- if (conditional) statement else statement
- while (conditional) statement
- break
- continue
- variable=expression
- print expression-list

Operatori aritmetici

- + somma
- - sottrazione
- * prodotto
- ** esponente
- / somma

Sono disponibili anche in versione "assegnativa" come in C (x+=y).

Operatori booleani

- && congiunzione
- || disgiunzione
- ! negazione

Predicati

- `< <= > >=` ordinamento (`3<2,"ciao"<="delta"`)
- `==` uguaglianza
- `!=` diversità
- `~` match con regex
- `!~` match negato con regex
- `in` chiave in array (`if (2 in vect) print vect[2]`)

Variabili

Ogni riferimento ad una variabile non definita comporta la creazione della stessa e la sua inizializzazione a "".

Esistono alcune variabili predefinite:

- **FS** (*Field Separator*) Identifica il separatore di input (Di default spazi o tab)
- **OFS** (*Output Field Separator*) Identifica il separatore di output (Di default lo spazio)
- **ORS** (*Output Record Separator*) Identifica il separatore di output di "record" (Di default `\n`)
- **NR** (*Number Record*) Contiene il numero di record processati, viene incrementata automaticamente. (*Read only*)
- **\$n** Identifica l'n-esimo campo del record corrente. `$0` è l'intero record.

È possibile formattare l'output come in C tramite l'utilizzo di `%d`, `%f` ecc.

È anche possibile utilizzare gli array come in C tramite la sintassi *variabile[indice]*.

Esempio di utilizzo *One-shot*

[Reg.Expr.]

```
# user@terminal: cat ordini.txt
pizza 7
patatine 2
# user@terminal: awk ' somma=somma+$2 END print somma ' ordini.txt
9
```

Esempio di utilizzo *Read-terminal*

```
# user@terminal: awk ' somma=somma+$2 END print somma '
# user@terminal: pizza 7
# user@terminal: patatine 2
[ctrl-d]
9
```

Esempio di utilizzo *Executable Scripts*

```
# user@terminal: nano test.awk
#!/bin/awk -f
# a sample awk program
{
    somma=somma+$2
}
END {print somma}
# user@terminal: chmod u+x test.awk
# user@terminal: ./test.awk ordini.txt
9
```

Esempio di utilizzo

Calcolo della lunghezza massima della riga di un file

```
# user@terminal: awk '{ if (length($0) > max ) max = length($0)} END { print $0 }
```

Esempio di utilizzo

Calcolo del numero di processi di un utente

```
# user@terminal: ps aux | awk '/^davide/ { tot++ } END { print "Totale processi: " tot }'
```

4.26 (*) fdisk - manipulate disk partition table

fdisk [*OPZIONI*] [*DISPOSITIVO*]

Richiede i permessi di superuser.

Permette di gestire e visualizzare la *partition table*

[Partizioni]

Esempio di utilizzo

```
# user@terminal: fdisk -l
```

4.27 ssh — OpenSSH remote login client

SSH, che sta per Secure Shell, è usato per connettersi in modo sicuro a un computer remoto. A differenza di Telnet, SSH è sicuro in quanto la connessione client/server è autenticata usando un certificato digitale e le password sono crittate. Pertanto è ampiamente usato dagli amministratori di sistema per controllare i server Linux remoti.

La sintassi per loggare in un Linux remoto è

ssh [*OPZIONI*] [*USERNAME*]@[*IP-ADDRESS/HOSTNAME*]

o in alternativa **ssh** [*OPZIONI*] [*USERNAME*]@[*IP-ADDRESS/HOSTNAME*] [*COMANDO*] [*PARAMETRI*]

Una volta loggato si può eseguire ogni comando supportato dall'host remoto. Per abilitare SSH sul tuo sistema Ubuntu, dovrai installare il server OpenSSH. Esegui il seguente comando nella finestra del terminale:

```
# user@terminal: sudo apt install openssh-server
```

La porta predefinita è 22, ma può essere cambiata. Per controllare lo stato del server ssh può essere utilizzato il comando:

```
# user@terminal: sudo systemctl status ssh
```

Il server SSH supporta differenti schemi di autenticazione. I due più popolari sono:

- Autenticazione basata su password
- Autenticazione basata su chiave pubblica

Passi per impostare le chiavi SSH:

1. Crea un paio di chiavi utilizzando il comando **ssh-keygen**.
2. Copia e installa la chiave pubblica utilizzando il comando **ssh-copy-id** sul server.
3. Aggiungiti te stesso al sudo o
4. Disabilita la password di login per l'account di root.
5. Testa la tua connessione senza password di SSH utilizzando il comando **ssh user@server-name**

Il file `$HOME/.ssh/id_rsa` contiene la tua chiave privata.

Il file `$HOME/.ssh/id_rsa.pub` contiene la tua chiave pubblica.

4.28 scp - OpenSSH secure file copy

fdisk [*OPZIONI*] [*SORGENTE*] [*DESTINAZIONE*]

Opzioni utilizzate

-r (*recursive*) copia ricorsivamente (utilizzato per copiare cartelle)

Permette di copiare in sicurezza file tra dispositivi in una rete.

Esempio di utilizzo per copiare da un host remoto al locale

```
# user@terminal: scp username@from_host:file.txt /local/directory/
```

Esempio di utilizzo per copiare dal local ad un host remoto

```
# user@terminal: scp file.txt username@to_host:/remote/directory/
```

4.29 (*) docker

[Teoria Docker]

4.29.1 pull

docker pull [*IMMAGINE*]

Permette scaricare un'immagine o un repository da un registro, come Docker Hub o altri. Con questo comando, puoi specificare il nome, il tag o il digest dell'immagine che vuoi ottenere.

Esempio di utilizzo

Per scaricare un hello world.

```
# user@terminal: docker pull hello-world
```

Per scaricare l'immagine ufficiale di Ubuntu dalla versione 20.04.

```
# user@terminal: docker pull ubuntu:20.04
```

4.29.2 run

docker run [*IMMAGINE*]

Permette di lanciare un immagine in un container.

Opzioni utilizzate

-it (*Interactive terminal*) Permette di interagire con il container.

-name Permette di assegnare un nome al container.

Esempio di utilizzo

```
# user@terminal: docker run hello-world
```

```
Hello from Docker!
```

This message shows that your installation appears to be working correctly.

4.29.3 Altri comandi

1

attach: Attach local standard input, output, and error streams to a running container
build: Build an image from a Dockerfile
commit: Create a new image from a container's changes
cp: Copy files/folders between a container and the local filesystem
create: Create a new container
diff: Inspect changes to files or directories on a container's filesystem
events: Get real time events from the server
exec: Run a command in a running container
export: Export a container's filesystem as a tar archive
history: Show the history of an image
images: List images
import: Import the contents from a tarball to create a filesystem image
info: Display system-wide information
inspect: Return low-level information on Docker objects
kill: Kill one or more running containers
load: Load an image from a tar archive or STDIN
login: Log in to a Docker registry
logout: Log out from a Docker registry
logs: Fetch the logs of a container
pause: Pause all processes within one or more containers
port: List port mappings or a specific mapping for the container
ps: List containers
pull: Pull an image or a repository from a registry
push: Push an image or a repository to a registry
rename: Rename a container
restart: Restart one or more containers
rm: Remove one or more containers
rmi: Remove one or more images
run: Run a command in a new container
save: Save one or more images to a tar archive (streamed to STDOUT by default)
search: Search the Docker Hub for images
start: Start one or more stopped containers
stats: Display a live stream of container(s) resource usage statistics
stop: Stop one or more running containers
tag: Create a tag TARGET_IMAGE that refers to SOURCE_IMAGE
top: Display the running processes of a container
unpause: Unpause all processes within one or more containers
update: Update configuration of one or more containers
version: Show the Docker version information
wait: Block until one or more containers stop, then print their exit codes

Esempio di installazione immagine da git

```
# user@terminal: git clone https://github.com/davide-dipierro/Duckerfile
# user@terminal: cd Duckerfile
# user@terminal: sudo docker build -t my-duck-ubuntu .
OUTPUT OMESSO
# user@terminal: sudo docker run my-duck-ubuntu
# user@terminal: duck
```



¹Copiato dall'`--help` di Docker

4.30 netstat

[Teoria Socket]

netstat [*OPZIONI*]

Permette di visualizzare le connessioni di rete, le tabelle di routing e una serie di informazioni relative alle interfacce di rete.

Opzioni utilizzate

- a** (*all*) mostra tutte le connessioni
- n** (*numeric*) mostra gli indirizzi numerici
- p** (*program*) mostra il programma che ha aperto la connessione
- t** (*tcp*) mostra solo le connessioni tcp
- u** (*udp*) mostra solo le connessioni udp
- l** (*listening*) mostra solo le connessioni in ascolto

Esempio di utilizzo

```
# user@terminal: netstat -tulpn
```

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	State	PID
tcp	0	0	127.0.0.53:53	0.0.0.0:*	LISTEN	-
udp	0	0	127.0.0.53:53	0.0.0.0:*		-
udp	0	0	127.0.0.1:323	0.0.0.0:*		-
udp	0	0	0.0.0.0:60498	0.0.0.0:*		-
udp6	0	0	:::1:323	:::*		-

Dischi e partizioni

Ogni disco rigido è generalmente suddiviso in una serie di unità separate di dimensioni diverse chiamate partizioni o sezioni.

Ogni disco contiene una qualche forma di tabella delle partizioni, chiamata VTOC (Volume Table Of Contents), che descrive dove iniziano le sezioni e quale è la loro dimensione.

Ciascuna sezione può quindi essere utilizzata per archiviare informazioni di bootstrap, un file system, spazio di swap o essere lasciata come partizione grezza per l'accesso al database o altro uso.

5.1 Partizioni

Il partizionamento divide un'unità disco in uno o più dischi logici.

Ogni partizione viene trattata come un disco separato con il proprio file system.

Le informazioni sulle partizioni vengono archiviate in una tabella delle partizioni.

Esistono 2 tipi di partizioni che possono essere create utilizzando l'utilità *fdisk*:

- Partizioni primarie
- Partizioni secondarie

Le partizioni primarie sono le prime partizioni create su un disco e contengono il sistema operativo e i file di avvio.

Può contenere il programma di caricamento (*loader*) di un sistema operativo. Il loader si trova in un settore (il primo) della partizione chiamato *boot sector* e può contenere anche dati.

Le partizioni logiche generalmente contengono dati, ma possono contenere anche il Loader di Linux (Windows non può essere avviato da una partizione logica).

Lo schema di partizionamento originale per i dischi rigidi dei PC consentiva solo quattro partizioni, chiamate partizioni primarie.

Per creare più di quattro partizioni, una di queste quattro partizioni può essere divisa in tante partizioni più piccole, chiamate partizioni logiche.

Quando una partizione primaria è suddivisa in questo modo, è nota come partizione estesa. Dentro una partizione estesa possiamo creare fino a un massimo di 62 partizioni logiche.

Lo spazio dedicato ad una partizione deve essere contiguo in memoria.

Windows e linux

Windows e Linux usano differenti identificazioni.

Windows segna le partizioni con lettere e le chiama unità - non necessariamente corrispondono a unità fisiche.

Linux usa tre lettere e una cifra per la identificazione, iniziando con 'h' per IDE ed 's' per unità SCSI/SATA.

La terza lettera segna il numero dell'unità, come vista dal BIOS, con a-d per primaria/secondaria master/slave per unità IDE ed un numero illimitato per unità SCSI/SATA, basato sulle limitazioni del controller. La cifra si riferisce al numero delle partizioni.

I numeri 1-4 sono usati per denominare le partizioni primarie, una delle quali può essere una partizione estesa, un contenitore di partizioni logiche. Le partizioni logiche sono sempre numerate a partire da 5 e oltre. Fisicamente le partizioni logiche possono essere minori del loro numero, dipende dal numero di partizioni primarie presenti nel disco.

Le partizioni sono contate separatamente per ogni disco fisico, come riconosciuto dal sistema.

Per visualizzare l'elenco delle partizioni è disponibile il comando *fdisk*

5.1.1 Tipi di tabelle e partizioni

La tabella delle partizioni è una tabella di quattro elementi contenuta nel master boot record di un disco rigido. Ciascun elemento di questa tabella occupa 16 byte e contiene le seguenti informazioni relative a una partizione del disco:

1. flag partizione attiva: specifica se da questa partizione dev'essere caricato il nucleo del sistema operativo (1 byte),
2. indirizzo in formato CHS (Cylinder-Head-Sector) del primo settore della partizione (3 byte),
3. tipo di partizione: specifica se la partizione è primaria, estesa, nascosta, di ripristino o vuota, ed eventualmente, il tipo di file system utilizzato (1 byte),
4. indirizzo in formato CHS dell'ultimo settore della partizione (3 byte),
5. numero di settori del disco che precedono la partizione (4 byte),
6. numero di settori del disco che compongono la partizione (4 byte).

GUID Partition Table (GPT) è parte dello standard Extensible Firmware Interface (EFI). L'EFI utilizza il GPT laddove il BIOS utilizza il Master Boot Record (MBR).

MBR è composto da:

- MBP (Master Boot Program) è un codice eseguibile che si trova nei primi 446 byte del Master Boot Record;
- MBT (Master Boot Table) è una tabella che si trova subito dopo l'MBP, di 64 byte; gli ultimi 2 byte sono riservati al magic number dell'MBR, che identifica la fine dello stesso

GPT utilizza l'indirizzamento a blocchi logici.

5.1.2 Vantaggi di più partizioni

Fondamentalmente Linux è in grado di funzionare correttamente con una partizione, quella di Root, ed una partizione di Swap per la memoria virtuale.

Per un uso domestico, questo schema è decisamente sconsigliabile; qualora però si volesse reinstallare Linux, tutti i dati in esso contenuti andrebbero perduti. Per questo possiamo creare delle partizioni dedicate per quei files a noi più importanti come i dati personali dei nostri utenti, o tutta la posta e le code di stampa. Reinstallando Linux e, montando la partizione nel filesystem, non si perderebbero i dati.

Per ottenere maggiori prestazioni, si consiglia di creare le partizioni in un ordine logico ossia partendo dalle partizioni con una maggior frequenza di accesso. Un esempio di partizionamento per un server potrebbe essere, nell'ordine, Swap, /usr, /var, e /home.

Il vantaggio di avere filesystem su partizioni separate è che le diverse parti del sistema operativo sono in qualche modo protette le une dalle altre.

Se i tuoi utenti hanno riempito /home, i programmi che scrivono file di registro in /var non sono interessati se /home e /var sono partizioni separate.

Se il tuo disco viene danneggiato, solo la partizione danneggiata sarà danneggiata.

Lo svantaggio è che, nella maggior parte dei casi, se erroneamente hai allocato troppo poco spazio su disco per una partizione, non puoi rubare spazio da /var per avere più spazio su /home una volta configurato il sistema. Unix supporta la memoria virtuale (Swap), permettendo cioè di utilizzare spazio su disco come se fosse un'espansione di memoria RAM e aumentando di conseguenza la dimensione massima di memoria utilizzabile.

Consigli sulle partizioni

- Windows richiede partizioni primarie.
- BSD e Solaris richiedono anch'essi partizioni primarie.
- Linux non richiede partizioni primarie e può essere installato su quelle logiche.
- Installate sempre prima i sistemi che richiedono partizioni primarie.
- Create sempre prima le partizioni prima di installare un sistema, pensando bene alle vostre esigenze.
- Non dimenticate le limitazioni di dimensione per filesystems vecchi (come FAT32)
- In linea di massima è consigliabile avere una memoria di Swap doppia rispetto alla RAM presente.

5.2 Linux

Un'installazione tipica di Linux (compreso di server X e un ambiente grafico completo) occupa intorno agli 800 MB più la partizione di swap.

Le pretese si fanno più modeste se non si preferiscono i classici (e più snelli Windows Manager) ai più diffusi KDE e GNOME.

Riguardo al tipo di partizioni da utilizzare (primarie o logiche) consiglio di creare una partizione per il boot come primaria, utilizzare le restanti primarie disponibili, per poi allocare il nostro spazio rimanente in partizioni logiche all'interno di una estesa.

Creare:

1. Una partizione primaria per `/boot` di 15 mb (attenzione che vecchie versioni di LILO non permettono il boot da partizioni che superano il 1024 cilindro per una limitazione del BIOS, ciò non È più un problema dalla versione 21.3),
2. Una per `/home` (siate generosi con questa partizione se avete molto spazio),
3. Una per `/usr` (anche questa merita una particolare attenzione, diciamo che un paio di GB sono abbondanti per ogni uso),
4. Una per `/` dove risiederanno tutti i restanti file (un GB è più che sufficiente).
5. Sulla dimensione della partizione di swap in genere ci si regola assegnando il doppio della RAM presente sul nostro PC (ma non superate i 128 MB)

Per la grandezza di ogni partizione dipenderà dalle vostre esigenze:

- se per esempio prevedete di creare molti account sul vostro PC, o di utilizzarlo in rete come server rendendo disponibile dello spazio web o ftp, allora conviene avere una partizione per le home directory abbastanza grande,
- mentre se pensate di installare molti programmi dovrete avere un occhio di riguardo per la partizione con mount point in `/usr`.

Virtualizzazione

Per virtualizzazione si intende la creazione di una versione virtuale di una risorsa normalmente fornita fisicamente.

Qualunque risorsa hardware o software può essere virtualizzata: sistemi operativi, server, memoria, spazio disco, sottosistemi.

È quindi possibile eseguire, su una sola macchina (detta sistema host), altri sistemi operativi (detti sistemi guest) simultaneamente, ad esempio mettendo a disposizione su un sistema Windows un sistema Linux.

Le motivazioni che spingono alla virtualizzazione di un sistema sono diverse, a seconda di chi si appresta a virtualizzare un sistema. Gli sviluppatori hanno la possibilità di creare degli ambienti di prova nei quali far girare i loro software senza intaccare l'integrità del loro sistema, racchiudendo quindi i loro test all'interno di un'area controllata e priva di rischi.

Nell'ambito server è possibile avere server diversi, che offrono servizi diversi, sulla medesima macchina fisica, risparmiando sull'acquisto di nuove macchine serve.

La virtualizzazione si ottiene introducendo un livello di indirectione tra la vista logica e quella fisica delle risorse.

Sistema virtuale Risorse HW/SW virtuali	Vista fisica su sistema reale
Tecnologie di virtualizzazione	
Sistema reale Risorse HW/SW reali	

6.1 Esempi

- **Astrazione:** in generale un oggetto astratto (risorsa virtuale) è la rappresentazione semplificata di un oggetto (risorsa fisica):

- Esibendo le proprietà significative per l'utilizzatore
- Nascondendo i dettagli realizzativi non necessari.

Esempio: I tipi di dato del C sono un'astrazione della rappresentazione binaria dei dati nella cella di memoria.

Il disaccoppiamento è realizzato dalle operazioni (interfaccia) con le quali è possibile utilizzare l'oggetto.

- **Linguaggi di Programmazione:** La capacità di portare lo stesso programma (scritto in un linguaggio di alto livello) su architetture diverse è possibile grazie alla definizione di una macchina virtuale in grado di interpretare ed eseguire ogni istruzione del linguaggio, indipendentemente dall'architettura del sistema (S.O. e HW).

Esempi:

- La Java Virtual Machine e altri vari interpreti.
- Anche i compilatori possono essere visti, sotto un certo aspetto, come una virtualizzazione.

- **Virtualizzazione a livello di processo:** I sistemi multitasking permettono la contemporanea esecuzione di più processi, ognuno dei quali dispone di una macchina virtuale (CPU, memoria, dispositivi I/O) dedicata. La virtualizzazione è realizzata dal kernel del sistema operativo.

- **Virtualizzazione di Sistema:** Una singola piattaforma hardware viene condivisa da più sistemi operativi, ognuno dei quali è installato su una diversa macchina virtuale.

Il disaccoppiamento è realizzato da un componente chiamato *Virtual Machine Monitor* (VMM o Hypervisor) il cui compito è consentire la condivisione da parte di più macchine virtuali su una singola piattaforma hardware. Ogni macchina virtuale è costruita oltre che dall'applicazione che in essa viene eseguita, anche dal sistema operativo utilizzato.

VM1	VM2	VM3
Applicazioni	Applicazioni	Applicazioni
Sistema operativo	Sistema operativo	Sistema operativo
Virtual machine monitor		
Hardware		

Il VMM è il mediatore unico nelle interazioni tra le macchine virtuali e l'hardware sottostante, che garantisce:

- isolamento tra le macchine virtuali
- stabilità del sistema

6.2 Tipologie di virtualizzazione

Esistono 3 tipi di virtualizzazione: *Hardware*, *Virtualizzazione completa* e *Paravirtualizzazione*.

Hardware

Questo concetto fa riferimento a tecnologie che consentono la fornitura di componenti hardware, facendo ricorso a soluzioni software, e questo indipendentemente da quella che è la loro base fisica.

Una VM (virtual machine), altro non è che un computer virtuale che con l'utente finale si comporta esattamente come farebbe un computer fisico dotato di un suo hardware e relativo sistema operativo. Le VM dunque, operano come “sistemi guest virtuali” su uno o più sistemi fisici, denominati host.

L'hypervisor stabilisce il livello di astrazione tra il sistema virtuale e la base fisica. Gli Hypervisor sono dei software che hanno la funzione di gestire le risorse hardware a disposizione: Cpu, Ram, periferiche, memoria di archiviazione ed altre, condividendole con più sistemi guest.

Virtualizzazione completa

Attraverso questa forma di virtualizzazione, l'hypervisor di ogni macchina virtuale riproduce integralmente un ambiente hardware. In questa maniera, ogni VM ha un suo quantitativo di risorse hardware rese disponibili dall'hypervisor, e può di conseguenza eseguire delle applicazioni.

L'hardware fisico di quello che è il sistema host invece, rimane del tutto nascosto all'S.O. guest.

L'approccio permette di fatto il funzionamento di sistemi guest che non sono modificati.

Paravirtualizzazione

Con la virtualizzazione completa viene fornito una soluzione hardware virtuale per ciascun VM, mentre con la paravirtualizzazione l'hypervisor rende disponibile solamente un'API, ovvero un'interfaccia di programmazione. Questo permette ai sistemi operativi guest di poter accedere all'hardware fisico di quello che è il sistema host. In termini di prestazioni assolute dunque, la paravirtualizzazione offre dei vantaggi rispetto alla virtualizzazione completa.

Dal punto di vista dell'utente finale, una virtual machine non presenta differenze di sorta rispetto ad un computer fisico. In altre parole, la virtualizzazione hardware fa riferimento alla possibilità di scegliere differenti server virtuali per più utenti sulla base di una piattaforma di elaborazione molto potente: il popolare concetto di hosting condiviso.

6.3 Virtual Machine Monitor

In generale il Virtual Machine Monitor (VMM) deve offrire alle diverse macchine virtuali le risorse (virtuali) che sono necessarie per il loro funzionamento:

- CPU
- Memoria
- Dispositivi I/O

Può essere collocato a due livelli:

- **VMM di sistema:** esegue direttamente sopra l'hardware del sistema host (xen).
- **VMM ospitati:** eseguiti come applicazioni sopra un S.O. esistente (vmware player, virtualbox).

La modalità di dialogo per l'accesso alle risorse fisiche tra macchina virtuale ed il VMM:

- **Virtualizzazione completa** (vmware): le macchine virtuali utilizzano la stessa interfaccia (istruzioni macchina) dell'architettura fisica.
- **Paravirtualizzazione** (xen): il VMM presenta un'interfaccia diversa da quella dell'architettura hardware.

6.4 VMM di sistema vs. VMM ospitati

VMM di sistema

HOST: piattaforma di base sulla quale si realizzano macchine virtuali. Comprende la macchina fisica, l'eventuale sistema operativo ed il VMM.

GUEST: la macchina virtuale. Comprende applicazioni e sistema operativo.

VM1		VM2		VM3		
Applicazioni		Applicazioni		Applicazioni		GUEST
Sistema operativo		Sistema operativo		Sistema operativo		
Virtual machine monitor						HOST
Hardware						

VMM ospitato

Il VMM viene installato come un'applicazione sopra un sistema operativo esistente, che opera nello spazio utente e accede all'hardware tramite le system call del S.O. su cui viene installato.

Caratteristiche:

- Più semplice l'installazione (come un'applicazione).
- Può fare riferimento al S.O. sottostante per la gestione delle periferiche e può utilizzare altri servizi del S.O. (es. scheduling, gestione delle risorse.).
- Peggiora la performance.

VM1		VM2		VM3		
Applicazioni		Applicazioni		Applicazioni		GUEST
Sistema operativo		Sistema operativo		Sistema operativo		
		Virtual Machine Monitor				
Sistema operativo						HOST
Hardware						

VPN e virtualizzazione

Un ultimo riferimento sulle tipologie di virtualizzazioni, che ricordiamo sono davvero numerose, riguarda le reti. Sempre più diffuse sono infatti le VPN, ovvero le Virtual Private Network, reti virtuali basate su reti fisiche. Le VPN vengono utilizzate per la realizzazione di connessioni sicure. Un esempio può essere quello del collaboratore che necessita di accedere da una postazione esterna alla rete aziendale.

Dato che internet è una rete pubblica, la protezione dei tuoi dati non è garantita, in questo caso si consiglia la

virtualizzazione.

Molte aziende produttrici di software offrono soluzioni di virtualizzazione tramite i processi di crittografia e autenticazione. In questo modo la connessione avviene in una rete privata.

6.5 Docker

[Comandi Docker]

Docker è una piattaforma open-source che permette di creare, distribuire e gestire applicazioni in ambienti isolati chiamati container.

Un container è un'unità software che contiene tutto ciò che serve per eseguire un'applicazione, come il codice, le librerie, le dipendenze e il sistema operativo. Un esempio di container (Ma non del software docker) è il Python Virtual Environment, che permette di eseguire programmi Python in un ambiente separato.

Docker consente di impacchettare le applicazioni in container standardizzati e portatili, che possono essere eseguiti su qualsiasi ambiente informatico.

Docker offre diversi vantaggi per lo sviluppo e il deployment di applicazioni, tra cui:

- **Flessibilità:** Docker permette di creare applicazioni modulari basate su microservizi, che sono più facili da sviluppare, testare e scalare.
- **Efficienza:** Docker riduce lo spreco di risorse, poiché i container condividono il kernel del sistema operativo e utilizzano solo le risorse necessarie.
- **Velocità:** Docker accelera il processo di delivery, poiché i container sono leggeri e rapidi da avviare e fermare.
- **Portabilità:** Docker facilita la migrazione delle applicazioni tra diversi ambienti, come lo sviluppo locale, il testing, la produzione e il cloud.
- **Consistenza:** Docker garantisce la riproducibilità delle applicazioni, poiché i container mantengono le stesse configurazioni e dipendenze in ogni ambiente.
- **Sicurezza:** Docker aggiunge uno strato di separazione tra l'host reale e l'ambiente su cui gira il programma permettendo di proteggere l'host reale.

Problemi con più versioni

Docker permette di risolvere il problema di avere più versioni della stessa libreria isolando il lancio di un programma in un ambiente in cui è presente solo la versione necessaria di una specifica libreria.

Elementi di Docker

Dockerfile

Un Dockerfile è un file di testo che contiene una serie di istruzioni per creare un'immagine Docker, ovvero un pacchetto software che contiene tutto il necessario per eseguire un'applicazione in un container.

Di seguito un esempio di Dockerfile.

```
FROM openjdk
RUN useradd -ms /bin/bash ojdk
RUN mkdir -p /home/ojdk/app/ && chown -R ojdk:ojdk /home/ojdk/app
WORKDIR /home/ojdk/app
COPY *.java ./ COPY junit-*/
USER ojdk
RUN javac -cp "junit-4.10.jar:." *.java
COPY --chown=ojdk:ojdk .
CMD ["java", "-cp", "junit-4.10.jar:.", "org.junit.runner.JUnitCore", "TestAdd", "TestSub"]
```

Sintassi del dockerfile:

- **FROM** L'immagine base da utilizzare
- **RUN** Esegue comandi mentre crea l'immagine
- **WORKDIR** Specifica la directory da cui sono lanciati i comandi
- **USER** Cambia l'utente
- **COPY** Copia i file
- **CMD** Esegue comandi quando il container è in esecuzione

Docker Registry

Il Docker Registry è dove le immagini di Docker sono conservate. Il Docker Registry può essere pubblico o privato, locale o nel cloud.

Un'organizzazione può decidere di conservare le proprie immagini all'interno del Docker Hub.

Docker Hub

Il hub Docker è il registro pubblico gestito da Docker (Simile a Git Hub), dove si possono trovare e scaricare le immagini ufficiali o create dalla comunità.

Docker Volume

Un Docker Volume è una cartella condivisa tra il container e il computer host, che permette di salvare e scambiare dati persistenti tra i container o tra il container e l'host. Un Docker Volume è indipendente dal file system del container e rimane anche dopo la cancellazione del container.

6.6 Docker compose

Docker Compose, un tool di gestione piuttosto versatile che rende l'adozione di Docker ancora più agevole e facilmente gestibile l'adozione di applicazioni di questo tipo. Docker Compose è un tool ufficiale installabile su Linux (incluso Raspberry Pi OS), Windows, macOS e che consente, in modo piuttosto pratico, di gestire i propri container salvandone la configurazione di istanziazione in un unico file di configurazione in formato YAML, per esempio:

```
version: '3'
services:
  db:
    build: ./mysql
    image: poc/mysql-for-wordpress
    volumes:
      - db_data:/var/lib/mysql
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: somewordpress
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wordpress
      MYSQL_PASSWORD: wordpress
  wordpress:
    depends_on:
      - db
    image: wordpress:latest
    ports:
      - "8000:80"
    restart: always
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: wordpress
      WORDPRESS_DB_PASSWORD: wordpress
      WORDPRESS_DB_NAME: wordpress
```

```
volumes:  
  db.data: {}
```

Questo file di configurazione, per esempio, istruisce Docker Compose sull'istanziamento di due container, db (basato sull'immagine del motore MySQL per Docker) e wordpress (basato sull'immagine del CRM WordPress per Docker) e altro. Docker Compose serve, per l'appunto, a gestire in modo immediato i propri container. I suoi comandi infatti consentono di:

- avviare, fermare, e riavviare le applicazioni istanziate;
- vedere lo stato dei servizi in esecuzione;
- consultare lo stream dei log dei servizi in esecuzione;
- eseguire comandi all'interno dei container
- altro

In pratica, senza l'adozione di Docker Compose ogni qual volta è necessario mettere in esecuzione un container, è necessario conoscerne il comando ed eseguirlo.

Usando Docker Compose affiancato a Docker, ogni qual volta si ha necessità di aggiungere un container al proprio "stack" è sufficiente aggiungere una porzione di codice di configurazione ad esso relativa al proprio file `docker-compose.yaml`, il quale consentirà a Docker Compose di "conoscere" le caratteristiche del nuovo container (oltre a quelli già presenti) e quindi consentirne una rapida e pratica gestione.

Per praticità.

Immaginiamo banalmente di dover amministrare un certo numero di container: anche solo pensando alla nostra amata domotica personale, volendo istanziare tutte le componenti necessarie in un sistema di media complessità, potremmo arrivare a definirne anche una decina.

Va da sé che per ognuno sarebbe necessario segnarsi da parte il corretto comando di esecuzione (in base ai tanti parametri diversi di ognuno) per eseguirlo nuovamente ogni qual volta sia necessario cancellare e ricreare il container (per esempio, in caso di aggiornamento dell'immagine per sopravvenuto aggiornamento dell'applicazione). Con Docker Compose, invece, una volta dichiarate nel suo file di configurazione le configurazioni dei vari container, di esse ci si può dimenticare: basterà infatti utilizzare i suoi comandi rapidi.

Spesso docker-compose può essere installato come plug-in di Docker (Oltre che come programma a sé).

Usare Compose è essenzialmente un processo di tre passi:

1. Definisci l'ambiente della tua app con un Dockerfile in modo che possa essere riprodotto ovunque.
2. Definisci i servizi che compongono la tua app in un file `compose.yaml` in modo che possano essere eseguiti insieme in un ambiente isolato.
3. Esegui `docker compose up` e il comando `Docker compose` avvia ed esegue la tua intera app.

Gestione dei dati

Per impostazione predefinita, tutti i file creati all'interno di un container sono memorizzati su uno strato scrivibile del container. Questo significa che:

- I dati non persistono quando il container non esiste più, e può essere difficile estrarre i dati dal container se un altro processo ne ha bisogno.
- Lo strato scrivibile di un container è strettamente accoppiato alla macchina host dove il container è in esecuzione. Non è possibile spostare facilmente i dati da qualche altra parte.
- Scrivere nello strato scrivibile di un container richiede un driver di archiviazione per gestire il filesystem. Il driver di archiviazione fornisce un'unione dei filesystem, utilizzando il kernel Linux. Questa ulteriore astrazione riduce le prestazioni rispetto all'uso di volumi di dati, che scrivono direttamente sul filesystem host.

Docker ha due opzioni per i contenitori per memorizzare i file sulla macchina host, in modo che i file siano persistenti anche dopo che il container si ferma: volumi e bind mounts.

Vantaggi di Docker compose

File Compose: l'utilizzo di un file YAML, tipicamente denominato `docker-compose.yml`, Docker Compose consente di definire i servizi, le reti e i volumi dell'applicazione. Questo file funge da modello completo per l'intera applicazione, semplificando la gestione, la riproduzione e la condivisione della configurazione dell'applicazione.

Servizi: In Docker Compose i servizi indicano singole istanze di container create da specifiche immagini Docker. Questi servizi sono la base di un'applicazione multi-container e possono essere interconnessi attraverso le reti, condividendo i dati con i volumi.

Reti: Docker Compose facilita la creazione di reti personalizzate per le applicazioni containerizzate, consentendo una comunicazione fluida tra i servizi. Sebbene Compose generi automaticamente una singola rete per tutti i servizi per impostazione predefinita, è possibile definire reti aggiuntive per isolare servizi specifici o stabilire topologie complesse.

Volumi: Docker Compose offre volumi per conservare i dati generati e utilizzati dai contenitori Docker. I volumi facilitano la condivisione dei dati tra i contenitori e garantiscono la conservazione dei dati quando un contenitore viene rimosso o aggiornato.

Scalabilità: Con Docker Compose è possibile scalare facilmente i servizi per gestire carichi di lavoro maggiori, regolando il numero di istanze del contenitore per un servizio specifico. Questo può essere fatto usando l'attributo `scale` o l'opzione `--scale` della riga di comando.

Sostituzione di variabili d'ambiente: Docker Compose supporta la sostituzione delle variabili d'ambiente, consentendo di adattare il file Compose a vari ambienti o fasi della pipeline di sviluppo. Questa funzione consente una maggiore flessibilità e personalizzazione in diversi scenari di distribuzione.

version: '3'	
services:	
db:	Backend service
image: mysql:5.7	
volumes:	Specify Volumes/Network
- db_data:/var/lib/mysql	
restart: always	
environment:	
MYSQL_ROOT_PASSWORD: somewordpress	Environmental variables
MYSQL_DATABASE: wordpress MYSQL_USER: wordpress	
MYSQL_PASSWORD: wordpress	
wordpress:	Frontend services
depends_on:	
- db	
image: wordpress:latest	Specify Volumes/Network
ports:	
- "8000:80"	
restart: always	
environment:	
WORDPRESS_DB_HOST: db:3306	Environmental variables
WORDPRESS_DB_USER: wordpress	
WORDPRESS_DB_PASSWORD: wordpress	
volumes:	
db_data:	

6.7 Esempio di docker-compose

Esegui i seguenti passaggi per creare un webserver Python su docker compose

Definire dipendenze

Crea una directory per il progetto

```
# user@terminal: mkdir composetest
# user@terminal: cd composetest
```

Crea un file chiamato app.py nella cartella del tuo progetto e inserisci il seguente codice

```
import time
import redis
from flask import Flask

app = Flask(__name__)
cache = redis.Redis(host='redis', port=6379)

def get_hit_count():
    retries = 5;
    while True:
        try:
            return cache.incr('hits')
        except redis.exception.ConnectionError as exc:
            if retries == 0:
                raise exc
            retries -= 1
            time.sleep(0.5)
@app.route('/')
def hello():
    count = get_hit_count()
    return 'Hello World! I have been seen {} times.\n'.format(count)
```

Crea un file requirements.txt e incolla il seguente codice

```
flask
redis
```

Creare Dockerfile

Creare un file chiamato Dockerfile ed incollare il seguente codice

```
# syntax=docker/dockerfile:1
FROM python:3.7-alpine
WORKDIR /code
ENV FLASK_APP=app.py
ENV FLASK_RUN_HOST=0.0.0.0
RUN apk add --no-cache gcc musl-dev linux-headers
COPY requirements.txt requirements.txt
RUN pip install -r requirements.txt
EXPOSE 5000
COPY . .
CMD ["flask", "run"]
```

Definire i servizi

Crea un file chiamato compose.yaml ed incolla il seguente codice

```
services:
  web:
    build: .
    ports:
      - "8000:5000"
  redis:
    image: "redis:alpine"
```

Build e Run

Esegui e verifica che il server sia in esecuzione.

```
# user@terminal: sudo docker-compose up
```

Verifica se all'indirizzo <http://localhost:8000/> ci sia la seguente frase:

Hello World! I have been seen 1 times
Ricaricando la pagina il contatore dovrebbe incrementare.

6.8 Ambiente di sviluppo ADE

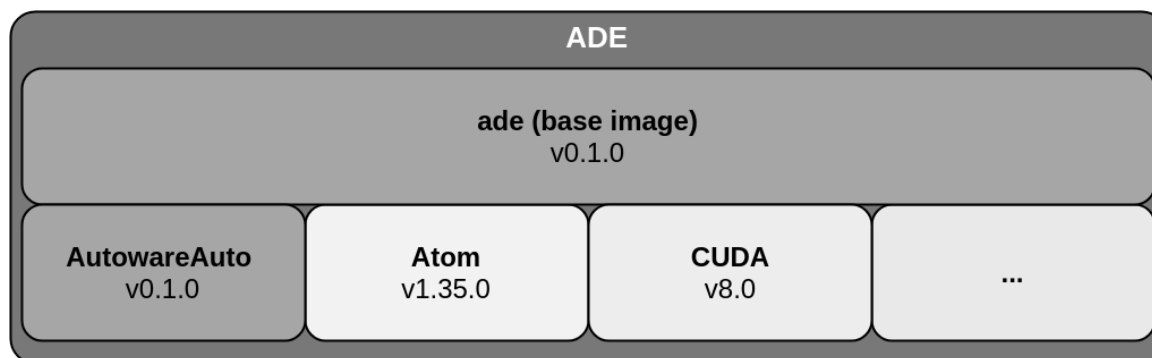
L'ambiente di sviluppo ADE è uno strumento modulare basato su Docker per garantire che tutti gli sviluppatori di un progetto dispongano di un ambiente di sviluppo comune e coerente.

Gli ADE gli sviluppatori:

- Sono certi che ciò che funziona su uno dei loro computer funzionerà anche su tutti i computer abilitati per ADE
- Possono installare comodamente nuovi pacchetti e dipendenze, sapendo che l'ambiente può essere facilmente riportato al suo stato originale
- Possono montare diversi volumi (che possono essere librerie autonome, IDE e altri strumenti) e facilmente passare da una versione all'altra di questi volumi senza influire su altre parti dell'ambiente

ADE crea un contenitore Docker da un'immagine di base e monta volumi di sola lettura aggiuntivi in `/opt`. L'immagine di base fornisce gli strumenti di sviluppo (ad esempio vim, udpreplay, ecc.) e i volumi forniscono strumenti di sviluppo aggiuntivi (ad esempio, IDE, librerie di terze parti di grandi dimensioni) o versioni software rilasciate (ad esempio l'ultima versione di AutowareAuto). Inoltre, ADE consente di passare facilmente da una versione all'altra di le immagini e i volumi.

Un esempio di ADE



L'immagine sopra mostra un esempio di una configurazione ADE. L'immagine di base, `ade`, viene utilizzata per creare un contenitore dove i volumi per AutowareAuto (una libreria software), Atom (un IDE) e CUDA (una grande libreria di terze parti) vengono montati come volumi.

6.8.1 Esempio di utilizzo di ADE

¹ Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Nunc aliquet bibendum enim facilisis gravida. Velit laoreet id donec ultrices tincidunt arcu non sodales neque. At risus viverra adipiscing at. In vitae turpis massa sed. Commodo elit at imperdiet dui accumsan sit. Pellentesque massa placerat duis ultricies lacus sed. Libero nunc consequat interdum varius sit amet mattis vulputate enim. Sed nisi lacus sed viverra tellus in hac. Vestibulum rhoncus est pellentesque elit ullamcorper dignissim. Sed egestas egestas fringilla phasellus faucibus scelerisque eleifend donec pretium. Viverra tellus in hac habitasse platea dictumst vestibulum. At erat pellentesque adipiscing commodo elit at imperdiet dui accumsan. Erat imperdiet sed euismod nisi. Elit duis tristique sollicitudin nibh sit. Lacus luctus accumsan tortor posuere ac ut consequat semper viverra. Nunc pulvinar sapien et ligula ullamcorper malesuada proin libero. Sit amet mattis vulputate enim. Felis imperdiet proin fermentum leo vel orci. Nibh sed pulvinar proin gravida hendrerit lectus.

¹L'utilizzo di ADE è un mistero per tutti, anche per gli sviluppatori di ADE stessi.

6.9 VAGRANT

Vagrant offre una soluzione astraendo le differenze di fondo tra le piattaforme VM, consentendo ai team DevOps di effettuare il provisioning di nuove VM con un'unica CLI e una sintassi di configurazione coerente.

I team che adottano Vagrant possono creare macchine virtuali coerenti indipendentemente dal fatto che vengano eseguite in un ambiente on-premise o in un ambiente cloud e possono passare facilmente da un ambiente all'altro. Vagrant permette inoltre ai team DevOps di migrare verso il cloud o da un provider all'altro senza dover riprogettare il processo di creazione delle macchine virtuali.

Le macchine virtuali create da Vagrant offrono un elevato livello di isolamento, spesso sfruttando il supporto specializzato per la virtualizzazione integrato nelle moderne CPU. Questo rende le macchine virtuali e gli strumenti che le creano, come Vagrant, la scelta migliore quando la sicurezza e l'isolamento sono una priorità assoluta.

Molte distribuzioni Linux, come Ubuntu e Fedora, forniscono anche box Vagrant ufficiali su cui i team DevOps possono costruire. Questo riduce il tempo necessario per creare macchine virtuali personalizzate.

Vantaggi

Spesso le macchine virtuali non sono sufficienti in quanto il provisioning non è tracciabile e quindi non reperibile. Vagrant invece consente di avere delle macchine virtuali completamente "scriptabili" sia in fase di configurazione (RAM, networking, spazio disco...) che di provisioning (installazione di MongoDB, RabbitMQ...). Ognuno di questi script potrà poi essere aggiunto al nostro source control preferito e arrivare a versionare i vari scenari realizzando quella pratica nota come Infrastructure as code.

Tra i vantaggi possiamo riassumere:

- capacità di virtualizzare la virtualizzazione (Ovvero la possibilità di gestire macchine virtuali da linea di comando in modo semplice e veloce).
- creare e gestire macchine virtuali da linea di comando in modo semplice e veloce
- condividere immagini di macchine virtuali
- lavorare sulla macchina virtuale via terminale per mezzo di SSH
- gestione semplificata di una cartella condivisa per editare file da host (e non da macchina virtuale).

Docker o Vagrant?

Vagrant permette di creare e gestire ambienti di macchine virtuali (VM) in un unico flusso di lavoro. Sia che si stiano creando delle macchine virtuali locali da eseguire con gli hypervisor o solo delle macchine virtuali per il cloud, Vagrant offre comunque un formato di configurazione coerente, un'unica CLI e dei provisioner condivisi per installare il software e modificare le configurazioni.

Docker offre la possibilità di impacchettare il software e la configurazione di supporto in immagini che vengono eseguite in modo coerente su più piattaforme. Inoltre, permette ai team DevOps di avere la certezza che il software venga eseguito su una workstation locale nello stesso modo in cui viene eseguito su una piattaforma cloud gestita.

Sebbene Vagrant e Docker condividano l'obiettivo di creare ambienti ripetibili, lo fanno in modi diversi ma complementari.

Vagrant e Docker non sono tecnologie che si escludono a vicenda ed entrambi gli strumenti possono essere utilizzati fianco a fianco. Ad esempio, i team DevOps possono utilizzare Docker per sviluppare ed eseguire applicazioni e Vagrant per ricreare ambienti specializzati per riprodurre problemi.

Docker può anche essere eseguito all'interno di una macchina virtuale creata da Vagrant, magari per testare nuove versioni di Docker o per testare strumenti distribuiti come immagini Docker in un ambiente isolato.

- I Dockerfile corrispondenti a Vagrant si chiamano **Vagrantfile**.
- I container corrispondenti di Vagrant si chiamano **Vagrant Boxes**.

Linguaggio C

7.1 Sintassi

Perché sei qui? Dovresti già conoscerla...¹

7.2 File

Il kernel di UNIX vede tutti i file come flussi non formattati di byte; il compito di interpretare ogni struttura logica interna ad un file lasciato è alle applicazioni.

I file UNIX sono strutture composte dai tre seguenti elementi:

1. **Nome:** Stringa di caratteri alfanumerici utilizzata da utenti e programmi per fare riferimento al file.
2. **Inodo:** Struttura dati che contiene le informazioni sul file necessarie al SO per la sua gestione (attributi), oltre agli indirizzi per accedere ai dati contenuti nel file.
3. **Dati:** I dati effettivamente contenuti nel file.

Il sistema assegna a ciascun file un identificatore numerico, detto i-node.

Modo del file	Flag di 16 bit dove sono memorizzati il tipo del file ed i suoi permessi di accesso (read, write, execute)
Contatore link	Numero di riferimenti all'inodo nelle directory
ID Proprietario	Identificatore del proprietario del file
ID Gruppo	Identificatore del gruppo a cui appartiene il proprietario
Dimensione	Dimensione in byte del file
Indirizzi	Informazioni di indirizzamento ai dati del file
Ultimo accesso	Tempo dell'ultimo accesso ai dati del file
Ultima modifica	Tempo dell'ultima modifica ai dati del file
Ultimo modifica stato	Tempo dell'ultima modifica all'inodo

7.2.1 Accesso/creazione di File

Quando un processo si riferisce ad un file per nome (path assoluto o relativo), il kernel suddivide il path per componente, controllando i permessi di accesso alle relative directory.

In caso di esito positivo, il kernel:

1. **Ritrova l'inodo** del file, se il file esiste e il processo chiede di accedervi;
2. **Assegna un inodo** non usato, se il processo chiede di creare un nuovo file.

In entrambi i casi il kernel restituisce al processo un intero non negativo detto, descrittore del file, che resta associato al file (attraverso il relativo inodo) fino a quando il file non viene rilasciato.

¹O in alternativa: <https://www.html.it/guide/guida-c/>

7.3 Operazioni sui file

La maggior parte delle operazioni sui file ordinari in ambiente UNIX si possono eseguire utilizzando solo le cinque chiamate di sistema: *open*, *read*, *write*, *lseek*, *close*. Quando un file viene aperto, il kernel restituisce al processo un file descriptor (numero intero). Di default:

- **stdin**: 0
- **stdout**: 1
- **stderr**: 2

open

```
int open(const char *pathname, int flags, mode_t mode);
```

Il *pathname* può essere relativo o assoluto.

Il parametro *flags* è un intero che può essere costruito utilizzando le seguenti costanti:

1. **O_RDONLY**: apre il file in sola lettura
2. **O_WRONLY**: apre il file in sola scrittura
3. **O_RDWR**: apre il file in lettura e scrittura
4. **O_CREAT**: crea il file se non esiste
5. **O_TRUNC**: tronca il file a zero se esiste
6. **O_APPEND**: scrive in coda al file

Il parametro *mode* è il modo del file ed è un parametro opzionale, utilizzato solo nel caso di creazione del file. La funzione restituisce il file descriptor del file aperto, oppure -1 in caso di errore.

Esempio di utilizzo di open

```
int fileDescriptor = open("file.txt", O_RDONLY);
```

create

```
int create(const char *pathname, mode_t mode);
```

Funziona allo stesso modo del comando *open*, ma se il file esiste già, viene troncato a zero.

La funzione restituisce il file descriptor del file aperto, oppure -1 in caso di errore.

Esempio di utilizzo di create

```
int fileDescriptor = create("file.txt", O_RDONLY);
```

read

```
ssize_t read(int fd, void *buf, size_t count);
```

Restituisce il numero di byte letti, oppure -1 in caso di errore.

L'operazione di lettura avviene a partire dalla posizione corrente del file, che viene incrementata del numero di byte letti.

Il numero di byte letti può essere inferiore quando:

- Il file contiene meno di *count* byte a partire dalla posizione corrente
- La lettura è stata interrotta da un segnale
- La lettura avviene da terminale in cui si legge una riga alla volta
- La lettura avviene da una socket, da una FIFO o da un pipe senza che sia stato scritto un numero sufficiente di byte per riempire il buffer

Esempio di utilizzo di read

```
char buffer[100];
int fileDescriptor = open("file.txt", O_RDONLY);
int n = read(fileDescriptor, buffer, 100);
```

write

```
ssize_t write(int fd, const void *buf, size_t count);
```

La funzione restituisce il numero di byte scritti, oppure -1 in caso di errore.

L'operazione di scrittura avviene a partire dalla posizione corrente del file, che viene incrementata del numero di byte scritti.

Esempio di utilizzo di write

```
char buffer[100];
int fileDescriptor = open("file.txt", O_RDONLY);
int n = write(fileDescriptor, buffer, 100);
```

close

```
int close(int fd);
```

La funzione restituisce 0 in caso di successo, oppure -1 in caso di errore.

Esempio di utilizzo di close

```
int fileDescriptor = open("file.txt", O_RDONLY);
int n = close(fileDescriptor);
```

lseek

```
off_t lseek(int fd, off_t offset, int whence);
```

La funzione restituisce la nuova posizione corrente del file, oppure -1 in caso di errore.

Il parametro *offset* è un intero che indica il numero di byte da spostare.

Il parametro *whence* può assumere i seguenti valori:

- **SEEK_SET**: l'offset è relativo all'inizio del file
- **SEEK_CUR**: l'offset è relativo alla posizione corrente del file
- **SEEK_END**: l'offset è relativo alla fine del file

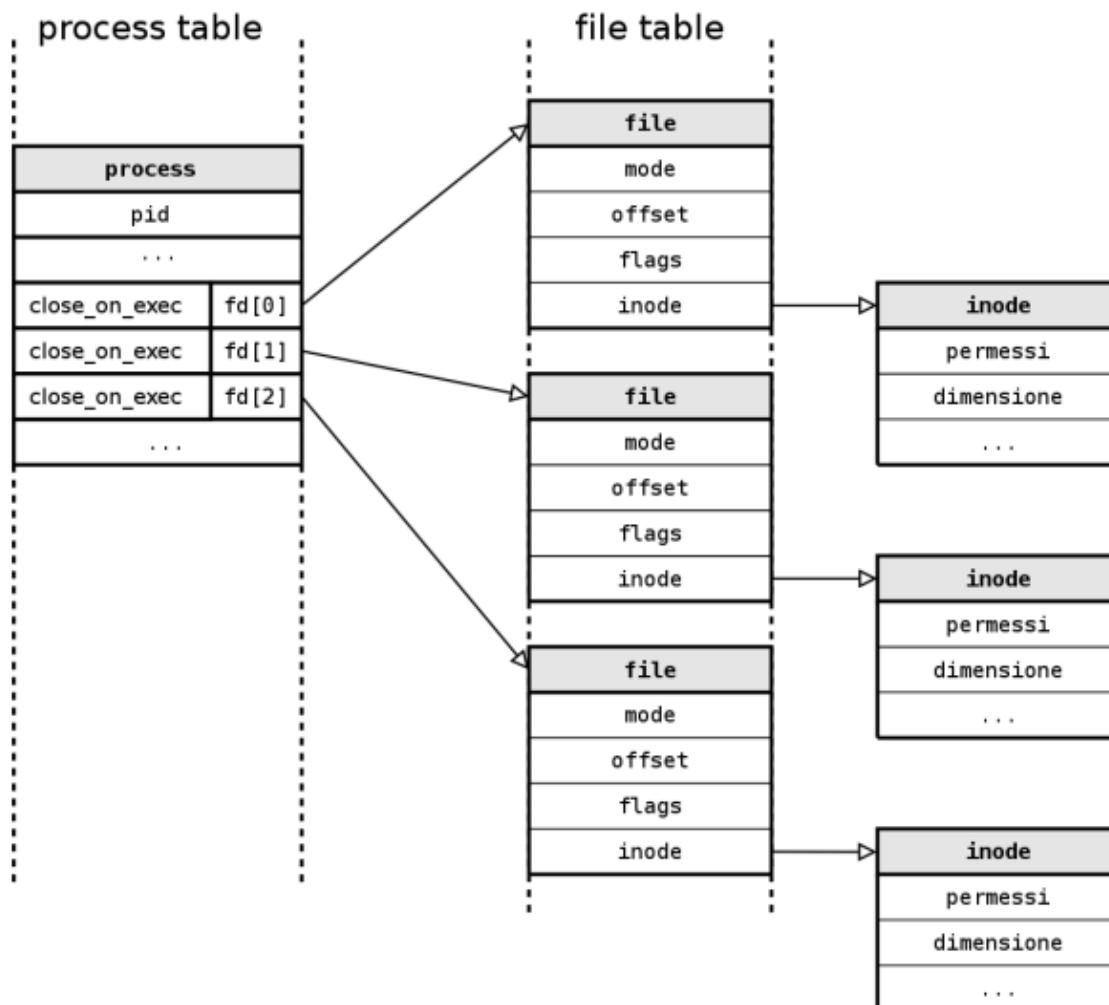
7.4 Implementazione nel kernel

Il kernel usa due strutture dati indipendenti per gestire i file aperti. Ogni processo mantiene una tabella dei propri file aperti (file descriptor table) che contiene un puntatore ad una entry nella tabella dei file aperti (file table) del kernel.

La file table specifica per ogni file aperto:

- **Posizione corrente**: posizione del file in cui avverrà la prossima operazione di lettura o scrittura
- **Contatore di riferimenti**: numero di processi che hanno aperto il file
- **Puntatore all'inodo**: puntatore all'inodo del file
- **Flags**: modalità di apertura del file

Ogni elemento della file table contiene un puntatore alla struttura dati *v-node* che contiene le informazioni sull'inodo del file.



7.5 Condivisione dei file

Il kernel utilizza tre strutture dati per la gestione dell' I/O:

- Ciascun processo ha un elemento nella tabella dei processi. Tale elemento è un "vettore" di descrittori di file aperti, ciascuno con: un puntatore ad un elemento della tabella dei file.
- Il kernel possiede una tabella per ciascun file aperto con i flag di stato del file (lettura, scrittura, append,...), l'offset corrente ed un puntatore ad un elemento della tabella dei v-node.
- Ciascun file aperto (o device) ha una struttura v-node. Il v-node contiene informazioni sul tipo di file e sulle funzioni che operano su di esso (informazione in i-node).

7.5.1 Esempi

Programma B

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <string.h>

int main() {
    char string[11];
    strncpy(string, "aaaaaaaaa\n", sizeof(string) - 1);
    string[sizeof(string) - 1] = '\0';
    int fd = open("testfile", O_RDWR | O_CREAT | O_APPEND, S_IRUSR | S_IWUSR);
    if (fd < 0) {
        perror("Errore in apertura");
        exit(1);
    }
}
```

```

}
char input;
do {
    if (write(STDOUT_FILENO, "Comando:", 8) < 8)
        perror("write error");
    input = getchar();
    fflush(stdin);
    string[0] = (char)input;
    write(fd, string, strlen(string));
    lseek(fd, (off_t)0, SEEK_SET); // sposta l'offset all'inizio del file
    if (write(STDOUT_FILENO, "Eseguito\n", 9) < 9)
        perror("write error");
} while (input != 'f');
close(fd);
return 0;
}

```

Programma A

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <string.h>

int main() {
    char string[11];
    strcpy(string, "bbbbbbbbb\n");
    int fd = open("testfile", O_RDWR | O_CREAT, S_IRUSR | S_IWUSR);
    if (fd < 0) {
        perror("Errore in apertura");
        exit(1);
    }
    lseek(fd, 0, SEEK_END); // sposta l'offset a FINE file
    int input;
    do {
        if (write(STDOUT_FILENO, "Comando:", 8) < 8)
            perror("write error su stdout");
        input = getchar();
        fflush(stdin);
        string[0] = (char)input;
        if (write(fd, string, strlen(string)) < strlen(string))
            perror("write error");
        if (write(STDOUT_FILENO, "Eseguito\n", 9) < 9)
            perror("write error su stdout");
    } while (input != 'f');
    close(fd);
    return 0;
}

```

7.5.2 Duplicazione di File descriptor

Un file descriptor può essere duplicato utilizzando:

```

#include <unistd.h>

int dup(int fd);
int dup2(int fd, int fd2);

```

La funzione *dup* restituisce il più piccolo file descriptor disponibile che punti allo stesso file del file descriptor *fd*.

La funzione *dup2* restituisce il file descriptor *fd2* se disponibile, altrimenti chiude *fd2* e restituisce il più piccolo file descriptor disponibile che punti allo stesso file del file descriptor *fd*.

7.6 Ottenere info su File

```

#include <sys/stat.h>
#include <unistd.h>

```

```
int stat(const char *file_name, struct stat *buf);
int fstat(int fd, struct stat *buf);
int lstat(const char *file_name, struct stat *buf);
```

Le funzioni *stat* e *lstat* restituiscono le informazioni sul file *file_name* nella struttura *buf*.

La funzione *fstat* restituisce le informazioni sul file associato al file descriptor *fd* nella struttura *buf*.

7.6.1 La struttura stat

```
struct stat {
    dev_t      st_dev;      /* ID del dispositivo contenente il file */
    ino_t      st_ino;      /* numero dell'inode */
    mode_t     st_mode;     /* protezione e tipo di file */
    nlink_t    st_nlink;    /* numero di hard links */
    uid_t      st_uid;      /* user ID del proprietario */
    gid_t      st_gid;      /* group ID del proprietario */
    dev_t      st_rdev;     /* ID del dispositivo (se il file speciale) */
    off_t      st_size;     /* dimensione totale, in byte */
    blksize_t  st_blksize;  /* dimensione dei blocchi di I/O del filesystem */
    blkcnt_t   st_blocks;   /* numero di blocchi assegnati */
    time_t     st_atime;    /* tempo dell'ultimo accesso */
    time_t     st_mtime;    /* tempo dell'ultima modifica */
    time_t     st_ctime;    /* tempo dell'ultimo cambiamento di stato */
};
```

Tipi di file st_mode

Il tipo di file associato al pathname è codificato nei bit più significativi del campo *st_mode* della struttura *stat*.

Tipo	Descrizione
S_IFMT	maschera per il tipo di file
S_IFSOCK	socket
S_IFLNK	link
S_IFREG	file regolare
S_IFBLK	dispositivo a blocchi
S_IFDIR	directory
S_IFCHR	dispositivo a caratteri
S_IFIFO	FIFO o pipe
S_ISUID	bit set-UID
S_ISGID	bit set-GID
S_ISVTX	sticky bit
S_IRWXU	maschera per i permessi del proprietario (lettura, scrittura e esecuzione)
S_IRUSR	permesso di lettura per il proprietario
S_IWUSR	permesso di scrittura per il proprietario
S_IXUSR	permesso di esecuzione per il proprietario
S_IRWXG	maschera per i permessi del gruppo (lettura, scrittura e esecuzione)
S_IRGRP	permesso di lettura per il gruppo
S_IWGRP	permesso di scrittura per il gruppo
S_IXGRP	permesso di esecuzione per il gruppo
S_IRWXO	maschera per i permessi di altri utenti (lettura, scrittura e esecuzione)
S_IROTH	permesso di lettura per altri utenti
S_IWOTH	permesso di scrittura per altri utenti
S_IXOTH	permesso di esecuzione per altri utenti

7.6.2 User e Group ID

Ad ogni utente è associato un identificatore numerico univoco, detto *user ID*.

Ad ogni gruppo è associato un identificatore numerico univoco, detto *group ID*.

Ad ogni file è associato un *user ID* ed un *group ID* che indicano rispettivamente il proprietario ed il gruppo proprietario del file.

Ogni processo possiede i seguenti ID:

- **Real user ID:** identifica l'utente che ha lanciato il processo
- **Effective user ID:** identifica l'utente che determina i diritti di accesso del processo
- **Real group ID:** identifica il gruppo dell'utente che ha lanciato il processo
- **Effective group ID:** identifica il gruppo dell'utente che determina i diritti di accesso del processo

7.6.3 Esempio di utilizzo

```
#include <sys/stat.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

int main() {
    struct stat info;
    if (stat("file.txt", &info) == -1) {
        perror("Errore in stat");
        exit(1);
    }
    printf("Dimensione: %ld\n", info.st_size);
    printf("Ultimo accesso: %ld\n", info.st_atime);
    printf("Ultima modifica: %ld\n", info.st_mtime);
    printf("Ultimo cambiamento di stato: %ld\n", info.st_ctime);
    printf("User ID: %d\n", info.st_uid);
    printf("Group ID: %d\n", info.st_gid);
    printf("Tipo di file: %d\n", info.st_mode & S_IFMT);
    printf("Permessi: %d\n", info.st_mode & 0777);
    return 0;
}
```

7.6.4 Permessi

Il campo `st_mode` della struttura `stat` contiene i permessi di accesso al file.

Per avere accesso ad un file è necessario:

- **Permessi di esecuzione:** per accedere ad una directory è necessario avere i permessi di esecuzione sulla directory stessa e su tutte le directory padre.
- **Permessi di lettura:** per accedere ad un file è necessario avere i permessi di lettura sul file stesso e su tutte le directory padre.
- Avere i permessi specifici per il file.

Per creare un file in una directory è necessario avere i permessi di scrittura sulla directory stessa.

Per cancellare un file è necessario avere i permessi di scrittura sulla directory stessa.

L'accesso ai file è regolato dalla seguente sequenza di controlli:

1. Se l'effective user del processo è 0 (root), l'accesso è permesso.
2. Se l'effective user del processo è uguale all'owner del file, l'accesso è permesso se il bit di permesso per l'owner è settato.
3. Se l'effective group del processo è uguale al group owner del file, l'accesso è permesso se il bit di permesso per il group owner è settato.
4. Altrimenti l'accesso è permesso se il bit di permesso per altri utenti è settato.

7.6.5 Funzione `access`

```
#include <unistd.h>

int access(const char *pathname, int mode);
```

La funzione `access` controlla se il file specificato da `pathname` è accessibile con le modalità specificate da `mode`. La funzione restituisce 0 se l'accesso è permesso, -1 altrimenti. Controlla i permessi di accesso in base all'effective user ID e all'effective group ID del processo.

Il parametro `mode` può essere costruito utilizzando le seguenti costanti:

- R_OK: controllo di lettura
- W_OK: controllo di scrittura
- X_OK: controllo di esecuzione
- F_OK: controllo di esistenza

Esempio di utilizzo di access

```
if (access("file.txt", R_OK) == 0) {
    printf("Il file e' leggibile\n");
} else {
    printf("Il file non e' leggibile\n");
}
```

7.6.6 Funzione chmod - fchmod

```
#include <sys/stat.h>

int chmod(const char *pathname, mode_t mode);
int fchmod(int fd, mode_t mode);
```

La funzione *chmod* imposta i permessi di accesso al file specificato da *pathname* in base al parametro *mode*.

La funzione *fchmod* imposta i permessi di accesso al file associato al file descriptor *fd* in base al parametro *mode*.

La funzione restituisce 0 in caso di successo, -1 altrimenti. Il parametro *mode* può essere costruito utilizzando le seguenti costanti:

- S_IRUSR: permesso di lettura per il proprietario
- S_IWUSR: permesso di scrittura per il proprietario
- S_IXUSR: permesso di esecuzione per il proprietario
- S_IRGRP: permesso di lettura per il gruppo
- S_IWGRP: permesso di scrittura per il gruppo
- S_IXGRP: permesso di esecuzione per il gruppo
- S_IROTH: permesso di lettura per altri utenti
- S_IWOTH: permesso di scrittura per altri utenti
- S_IXOTH: permesso di esecuzione per altri utenti

Esempio di utilizzo di chmod

```
if (chmod("file.txt", S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH) == -1) { // rw-r--r--
    perror("Errore in chmod");
    exit(1);
}
```

7.6.7 Funzione chown - fchown

```
#include <unistd.h>

int chown(const char *pathname, uid_t owner, gid_t group);
int fchown(int fd, uid_t owner, gid_t group);
```

La funzione *chown* imposta l'owner ed il group owner del file specificato da *pathname* in base ai parametri *owner* e *group*.

La funzione *fchown* imposta l'owner ed il group owner del file associato al file descriptor *fd* in base ai parametri *owner* e *group*.

La funzione restituisce 0 in caso di successo, -1 altrimenti. Un processo può modificare l'owner ed il group owner di un file solo se:

- L'effective user ID del processo è 0 (root)
- L'effective user ID del processo è uguale all'owner del file
- L'effective group ID del processo è uguale al group owner del file

Esempio di utilizzo di chown

```
if (chown("file.txt", 1000, 1000) == -1) { // rw-r--r--
    perror("Errore in chown");
    exit(1);
}
```

Processi

8.1 Funzione main

Un programma C inizia la sua esecuzione obbligatoriamente dalla funzione `main`, che può essere definita in due modi:

- `int main(void)`: la funzione non accetta parametri e restituisce un intero
- `int main(int argc, char *argv[])`: la funzione accetta due parametri, il primo è il numero di argomenti passati al programma, il secondo è un array di stringhe contenente gli argomenti passati al programma

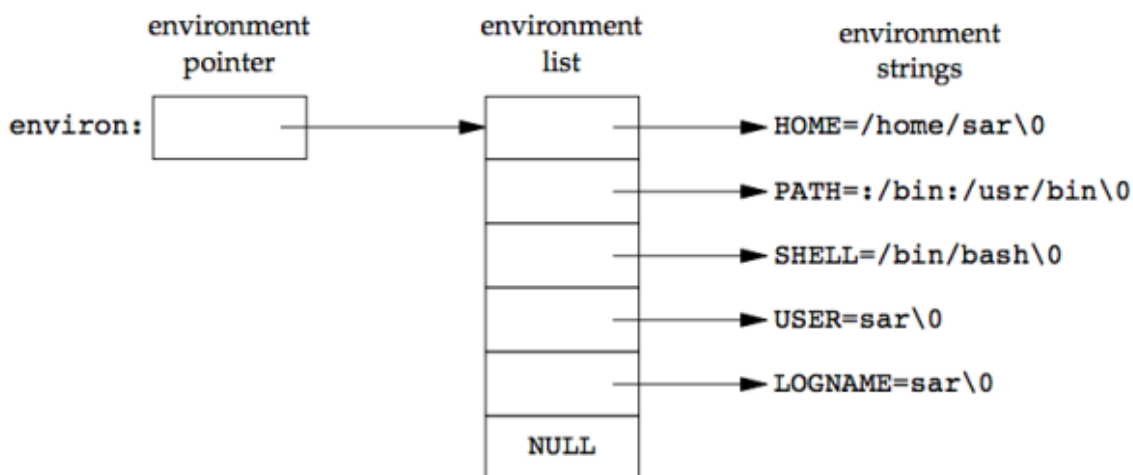
Lista di Ambiente

Ad ogni programma viene passata una lista di ambiente, che contiene variabili d'ambiente e i loro valori.

Analogamente alla lista degli argomenti, la lista di ambiente è un array di stringhe, ma ogni stringa ha la forma `nome=valore` con l'aggiunta di un elemento `NULL` alla fine dell'array.

Per accedere alla lista di ambiente è possibile utilizzare la variabile globale `environ`, che è un puntatore a un array di stringhe.

Le due funzioni `getenv` e `putenv` permettono di accedere alla lista di ambiente e di modificarla.

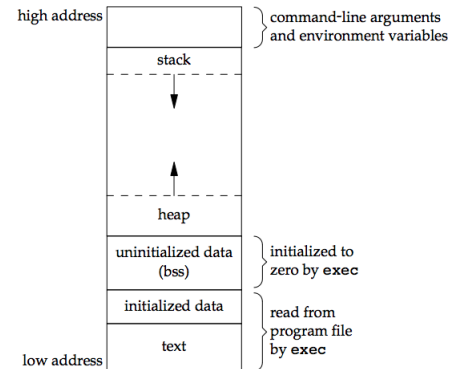


La funzione `getenv` accetta come parametro il nome di una variabile d'ambiente e restituisce il suo valore, mentre la funzione `putenv` accetta come parametro una stringa con la forma `nome=valore` e aggiunge la variabile d'ambiente alla lista di ambiente.

8.2 Processi

Layout della memoria

- **Testo:** contiene il codice del programma
- **Dati inizializzati:** contiene le variabili globali e statiche inizializzate
- **Dati non inizializzati:** contiene le variabili globali e statiche non inizializzate
- **Heap:** contiene le variabili allocate dinamicamente
- **Stack:** contiene le variabili locali e i parametri delle funzioni



Questo segmento dell'area di memoria riservata ad un programma serve a memorizzarne tutte le informazioni necessarie all'esecuzione.

Esecuzione di un programma

Quando un programma viene eseguito (tramite la funzione **exec**), una routine di start-up prende i command-line arguments e le environment variables e le mette in memoria.

La fine dell'esecuzione di un programma può avvenire in diversi modi:

- **Normal exit:** il programma termina normalmente, restituendo un valore alla funzione **main**
- **Error exit:** il programma termina normalmente, restituendo un valore diverso da 0 alla funzione **main**
- **Exit:** il programma termina normalmente, restituendo un valore alla funzione **exit**
- **Abort:** il programma termina in modo anomalo, restituendo un valore alla funzione **abort**
- **Signal:** il programma termina in modo anomalo, a causa di un segnale

Controllo processi

Il controllo dei processi in UNIX si esplica mediante:

- L'identificazione dei processi:
 - **pid_t getpid(void):** restituisce il PID del processo chiamante
 - **pid_t getppid(void):** restituisce il PID del processo padre del processo chiamante
 - **uid_t getuid(void):** restituisce l'UID del proprietario del processo chiamante
 - **uid_t geteuid(void):** restituisce l'UID effettivo del proprietario del processo chiamante
 - **gid_t getgid(void):** restituisce il GID del proprietario del processo chiamante
 - **gid_t getegid(void):** restituisce il GID effettivo del proprietario del processo chiamante
- La creazione di processi:
 - **pid_t fork(void):** crea un processo figlio
 - **pid_t vfork(void):** crea un processo figlio, che condivide lo spazio di indirizzamento con il processo padre
 - **pid_t clone(void):** crea un processo figlio, che condivide lo spazio di indirizzamento con il processo padre
 - **pid_t exec(void):** sostituisce il codice del processo chiamante con quello del programma passato come parametro
 - **int exit(int status):** termina il processo chiamante, restituendo un valore
 - **int wait(int *status):** attende la terminazione di un processo figlio

8.3 Identificazione dei processi

I processi in UNIX si dividono in processi di sistema e processi utente.

Ogni processo è identificato da un PID (Process Identifier), che è un intero positivo.

Il PID 0 è riservato al processo padre di tutti i processi.

Il PID 1 è riservato al processo init, che è il primo processo ad essere eseguito dal kernel.

Il PID 2 è riservato al processo pager, che è il processo che gestisce la memoria virtuale.

Ogni processo utente eredita una serie di altri identificativi:

- **UID:** User Identifier, identifica l'utente proprietario del processo
- **EUID:** Effective User Identifier, identifica l'utente effettivo proprietario del processo
- **GID:** Group Identifier, identifica il gruppo proprietario del processo
- **EGID:** Effective Group Identifier, identifica il gruppo effettivo proprietario del processo
- **PPID:** Parent Process Identifier, identifica il processo padre del processo

8.4 Creazione di processi

La creazione di un processo avviene tramite la funzione **fork** (o in alternativa (**vfork**)), che crea un processo figlio.

La funzione **fork** restituisce il PID del processo figlio al processo padre e 0 al processo figlio.

Il processo figlio eredita lo spazio di indirizzamento del processo padre.

Esempio

```
#include <stdio.h>
#include <unistd.h>

int main(void) {
    pid_t pid;
    pid = fork();
    if (pid == 0) {
        printf("Sono il processo figlio\n");
    } else {
        printf("Sono il processo padre\n");
    }
    return 0;
}
```

Padri e figli

Un processo può avere più processi figli, ma solo un processo padre.

Il figlio procede la propria esecuzione indipendentemente da quella del padre.

Il figlio ottiene una copia nuova della memoria del padre (variabili locali e globali).

Per ogni segnale, il figlio continua ad avere la stessa reazione del padre.

I file aperti dal padre sono ereditati dal figlio, i descrittori vengono copiati come con **dup**. In particolare, padre e figlio, condividono lo stesso file offset.

Funzione **fork** e **vfork**

La funzione **fork** crea un processo figlio, che eredita lo spazio di indirizzamento del processo padre.

La funzione **vfork** crea un processo figlio, che condivide lo spazio di indirizzamento con il processo padre.

La funzione **vfork** è più veloce della funzione **fork**, ma è più pericolosa, in quanto il processo figlio può modificare lo spazio di indirizzamento del processo padre.

Esempio di utilizzo di vfork

```
#include <stdio.h>
#include <unistd.h>

int main(void) {
    pid_t pid;
    pid = vfork();
    if (pid == 0) {
        printf("Sono il processo figlio\n");
        _exit(0);
    } else {
        printf("Sono il processo padre\n");
    }
    return 0;
}
```

È necessario introdurre la funzione `_exit`, che termina il processo figlio in modo da non interferire con il processo padre.

Race condition

Una race condition si verifica quando due o più processi accedono contemporaneamente alla stessa risorsa, senza sincronizzazione. L'ordine di esecuzione dei processi non è deterministico, quindi non è possibile prevedere il risultato dell'esecuzione.

Una chiamata ad una funzione `fork` o `fork` può causare race condition se le risorse del processo padre vengono modificate dal processo figlio o viceversa.

Esempio di race condition

```
#include <stdio.h>
#include <unistd.h>

int main(void) {
    int x = 0;
    pid_t pid;
    pid = vfork();
    if (pid == 0) {
        x = 1;
        printf("Sono il processo figlio, x = %d\n", x);
    } else {
        x = 2;
        printf("Sono il processo padre, x = %d\n", x);
    }
    return 0;
}
```

Esecuzione di programmi

L'esecuzione di un nuovo programma in UNIX si ottiene con la chiamata ad una delle funzioni `exec`.

Le funzioni `exec` non generano un nuovo processo, ma sostituiscono il codice del processo chiamante con quello del programma passato come parametro.

Quando un processo chiama una delle funzioni `exec`, le aree di memoria `text`, `data` e `stack` vengono sostituite con quelle del nuovo programma.

Il processo chiamante eredita la lista di ambiente del processo padre.

La famiglia di funzioni exec

La famiglia di funzioni `exec` è composta da:

- `int execl(const char *path, const char *arg0, ..., const char *argn, (char *)0)`: sostituisce il codice del processo chiamante con quello del programma passato come parametro, passando gli argomenti come parametri della funzione

- `int execl(const char *path, char *const argv[]):` sostituisce il codice del processo chiamante con quello del programma passato come parametro, passando gli argomenti come array di stringhe
- `int execlp(const char *path, const char *arg0, ..., const char *argn, (char *)0, char *const envp[]):` sostituisce il codice del processo chiamante con quello del programma passato come parametro, passando gli argomenti come parametri della funzione e la lista di ambiente come array di stringhe
- `int execve(const char *path, char *const argv[], char *const envp[]):` sostituisce il codice del processo chiamante con quello del programma passato come parametro, passando gli argomenti come array di stringhe e la lista di ambiente come array di stringhe
- `int execlp(const char *file, const char *arg0, ..., const char *argn, (char *)0):` sostituisce il codice del processo chiamante con quello del programma passato come parametro, passando gli argomenti come parametri della funzione, cercando il programma nella lista di directory specificata dalla variabile d'ambiente `PATH`
- `int execvp(const char *file, char *const argv[]):` sostituisce il codice del processo chiamante con quello del programma passato come parametro, passando gli argomenti come array di stringhe, cercando il programma nella lista di directory specificata dalla variabile d'ambiente `PATH`

Esempio di utilizzo di exec

```
#include <stdio.h>
#include <unistd.h>

int main(void) {
    printf("Sono il processo padre\n");
    execl("/bin/ls", "ls", "-l", (char *)0);
    printf("Questa istruzione non verra' mai eseguita\n");
    return 0;
}
```

Proprietà ereditate con exec

Quando un processo chiama una delle funzioni `exec`, le proprietà del processo chiamante vengono ereditate dal nuovo processo.

In particolare, il nuovo processo eredita:

- process ID e parent process ID
- real uid e real gid
- supplementary gid
- process group ID
- session ID
- terminale di controllo
- current working directory
- root directory
- maschera creazione file (umask)
- file locks
- maschera dei segnali
- segnali in attesa

Passare l'ambiente ad exec

L'ambiente di un processo è un insieme di stringhe (terminate da `\0`).

Un ambiente è rappresentato da un vettore di puntatori a caratteri terminato da un puntatore nullo.

Ogni puntatore (che non sia quello nullo) punta ad una stringa della forma: identificatore = valore

Per accedere all'ambiente da un programma C, è sufficiente aggiungere il parametro `envp` a quelli del `main`:

```
/* showmyenv */
#include <stdio.h>
main(int argc, char **argv, char **envp)
{
    while(*envp) printf("%s\n",*envp++);
}
```

oppure usare la variabile globale seguente: `extern char **environ`

L'ambiente di default di un processo coincide con quello del processo padre.

Per specificare un nuovo ambiente è necessario usare una delle due varianti seguenti della famiglia `exec`, memorizzando in `envp` l'ambiente desiderato:

```
execle(path, arg0, arg1, ..., argn, (char *)0, envp);
execve(path, argv, envp);
```

Esiste una funzione della libreria standard che consente di cercare in `environ` il valore corrispondente ad una specifica variabile d'ambiente:

```
#include <stdlib.h>
char *getenv(const char *name);
```

Current working directory e root directory

Il current working directory è la directory di lavoro del processo.

La root directory è la directory di partenza per le operazioni di ricerca di file.

Il current working directory e la root directory sono ereditati dal processo padre.

La chiamata di sistema seguente consente di cambiare la working directory di un processo:

```
#include <unistd.h>
int chdir(const char *path);
```

La chiamata di sistema seguente consente di cambiare la root directory di un processo:

```
#include <unistd.h>
int chroot(const char *path);
```

8.5 Terminazione di un processo

Quando un processo termina viene inviato il segnale `SIGCHLD` al padre.

Il processo diventa uno “zombie” finchè il padre non chiama una `wait/waitpid`.

Esistono tre modi per terminare un processo in modo normale:

- `int exit(int status)`: termina il processo chiamante, restituendo un valore
- `void _exit(int status)`: termina il processo chiamante, restituendo un valore
- `return`: termina la funzione `main`, restituendo un valore

Esistono due modi per terminare un processo in modo anomalo:

- `void abort(void)`: termina il processo chiamante, inviando il segnale `SIGABRT`
- Ricevendo determinati segnali.

Processi zombie

Cosa succede se il padre termina prima del figlio?

- Il figlio viene ereditato dal processo init che vuole evitare che il figlio diventi orfano (senza PPID).

Cosa succede se il figlio termina prima del padre?

- Generalmente il padre aspetta mediante la chiamata di sistema `wait` o `waitpid`.
- Se il padre non aspetta, il figlio diventa uno zombie.

Aspettare un figlio

La chiamata di sistema `pid_t wait(int *status)` permette di aspettare la terminazione di un processo figlio. La chiamata di sistema `pid_t waitpid(pid_t pid, int *status, int options)` permette di aspettare la terminazione di un processo figlio specifico.

La variabile `pid` contiene il PID del processo figlio.

La variabile `status` contiene il valore restituito dal processo figlio e può essere utilizzata con le seguenti macro:

- `WIFEXITED(status)`: restituisce 1 se il processo figlio è terminato normalmente, 0 altrimenti
- `WIFSIGNALED(status)`: restituisce 1 se il processo figlio è terminato a causa di un segnale, 0 altrimenti
- `WIFSTOPPED(status)`: restituisce 1 se il processo figlio è stato fermato da un segnale, 0 altrimenti
- `WIFCONTINUED(status)`: restituisce 1 se il processo figlio è stato ripristinato da un segnale, 0 altrimenti

La variabile `options` contiene le opzioni di esecuzione della funzione `waitpid`:

- `WNOHANG`: la funzione `waitpid` non attende la terminazione del processo figlio
- `WUNTRACED`: la funzione `waitpid` attende la terminazione del processo figlio, anche se è stato fermato da un segnale
- `WCONTINUED`: la funzione `waitpid` attende la terminazione del processo figlio, anche se è stato ripristinato da un segnale

Esempio di utilizzo di wait

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    pid_t pid;
    int status;
    pid = fork();
    if (pid == 0) {
        printf("Sono il processo figlio\n");
        _exit(0);
    } else {
        printf("Sono il processo padre\n");
        wait(&status);
    }
    return 0;
}
```

Esempio di utilizzo di waitpid

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    pid_t pid;
    int status;
    pid = fork();
    if (pid == 0) {
        printf("Sono il processo figlio\n");
```



```

        _exit(0);
    } else {
        printf("Sono il processo padre\n");
        waitpid(pid, &status, 0);
    }
    return 0;
}

```

La funzione system

```

#include <stdlib.h>
int system(const char *command);

```

La funzione `system` crea un processo figlio, che esegue il comando passato come parametro, e attende la sua terminazione.

Restituisce il valore restituito dal comando eseguito.

Esempio di utilizzo di system

```

#include <stdio.h>
#include <stdlib.h>

int main(void) {
    system("ls -l");
    return 0;
}

```

8.6 Segnali

Un segnale è un "interrupt" software che viene inviato ad un processo per notificare un evento.

La terminologia corretta è "exception", mentre "interrupt" è utilizzato per gli interrupt hardware.

Consente la comunicazione asincrona tra processi e/o tra device e processo. Ogni segnale ha un proprio nome che inizia con SIG.

I segnali sono identificati da un numero intero positivo.

Definiti nella libreria `signal.h`.

Generazione di un segnale

Segnali hardware

I segnali hardware sono generati da un dispositivo hardware.

Esempi:

- SIGBUS: errore di accesso alla memoria
- SIGFPE: errore di calcolo
- SIGILL: istruzione illegale
- SIGSEGV: violazione di segmento
- SIGSYS: chiamata di sistema illegale
- SIGTRAP: trap
- ...

Segnali software

I segnali software sono generati da un processo.

Esempi:

- **SIGALRM**: scadenza di un timer
- **SIGCHLD**: terminazione di un processo figlio
- **SIGCONT**: ripristino di un processo
- **SIGINT**: interruzione da tastiera
- **SIGKILL**: terminazione forzata
- **SIGPIPE**: scrittura su pipe senza lettori
- **SIGQUIT**: interruzione da tastiera
- **SIGSTOP**: arresto di un processo
- **SIGTERM**: terminazione di un processo
- **SIGTSTP**: interruzione da tastiera
- **SIGTTIN**: scrittura su pipe senza lettori
- **SIGTTOU**: scrittura su pipe senza lettori
- **SIGUSR1**: segnale utente 1
- **SIGUSR2**: segnale utente 2
- ...

Gestione dei segnali

Un processo può gestire i segnali in tre modi:

Ignorare il segnale

Il segnale viene ignorato e non viene eseguita alcuna azione.

I segnali **SIGKILL** e **SIGSTOP** non possono essere ignorati.

Catturare il segnale

Il segnale viene catturato e viene eseguita una funzione di gestione.

La funzione di gestione deve essere registrata con la funzione **signal** o **sigaction**.

La funzione di gestione viene eseguita in un nuovo processo, quindi non può accedere alle variabili locali del processo chiamante.

La funzione di gestione può essere eseguita in qualsiasi momento, anche durante l'esecuzione di un'altra funzione.

Esempio di utilizzo di signal

```
#include <stdio.h>
#include <signal.h>

void gestore(int sig) {
    printf("Sono il gestore del segnale %d\n", sig);
}

int main(void) {
    signal(SIGINT, gestore);
    while(1);
    return 0;
}
```

Quando il processo riceve il segnale **SIGINT**, viene eseguita la funzione **gestore**.

Inviare segnali

```
int kill(pid_t pid, int sig);
```

Un processo può inviare un segnale ad un altro processo con la funzione `kill`.
La funzione `kill` accetta due parametri:

- `pid_t pid`: PID del processo destinatario
- `int sig`: numero del segnale da inviare

Esempio di utilizzo di kill

```
#include <stdio.h>
#include <signal.h>

int main(void) {
    kill(1234, SIGINT);
    return 0;
}
```

Il processo invia il segnale `SIGINT` al processo con PID 1234.

Impostare una sveglia

```
unsigned int alarm(unsigned int seconds);
```

La funzione `alarm` permette di impostare una sveglia.

La funzione `alarm` accetta un parametro `seconds` che indica il numero di secondi dopo i quali deve essere inviato il segnale `SIGALRM`.

Se la funzione `alarm` viene chiamata più volte, il tempo viene aggiornato (non viene impostata una nuova sveglia).

Esempio di utilizzo di alarm

```
#include <stdio.h>
#include <signal.h>

void gestore(int sig) {
    printf("Sono il gestore del segnale %d\n", sig);
}

int main(void) {
    signal(SIGALRM, gestore);
    alarm(5);
    while(1);
    return 0;
}
```

Il processo invia il segnale `SIGALRM` dopo 5 secondi. Quando il processo riceve il segnale `SIGALRM`, viene eseguita la funzione `gestore`.

Pipe

9.1 Interprocess Communication (IPC)

Per cooperare, i processi hanno bisogno di comunicare.
I segnali sono un primo modo di farlo:

- Il messaggio consiste nel "numero del segnale" ed, eventualmente, le informazioni in `siginfo_t`.

Vogliamo però trasmettere informazioni arbitrarie e ben strutturate.

9.2 Pipe

Una pipe è un canale di comunicazione tra due processi fornendo un meccanismo attraverso il quale l'output di un processo diviene l'input per un altro processo. Il più semplice esempio di questo meccanismo è l'operatore `|` della shell.

Esempio

`ls | grep old`

è una pipeline grazie alla quale l'output del comando `ls` diventa l'input del comando `grep` senza un file intermedio di appoggio.

Caratteristiche

- Una pipe è un canale di comunicazione tra due processi.
- Una pipe ha un lato di scrittura ed uno di lettura.
- Una pipe è unidirezionale.
- Una pipe è un buffer circolare di byte.
- Una pipe ha una capacità massima.
- Una pipe è un file speciale.
- Una pipe (classica) necessita che i processi abbiano un antenato in comune.
- Una pipe è chiusa quando tutti i processi che la usano sono terminati.

9.3 La funzione Pipe

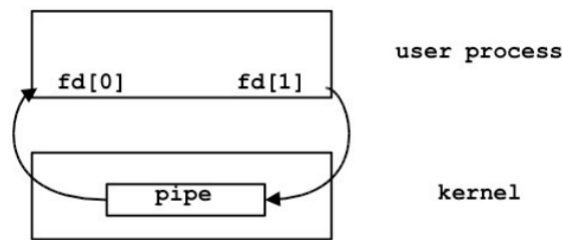
Creazione di una pipe

```
#include <unistd.h>
int pipe(int fd[2]);
```

- `fd[0]` è il descrittore di lettura.

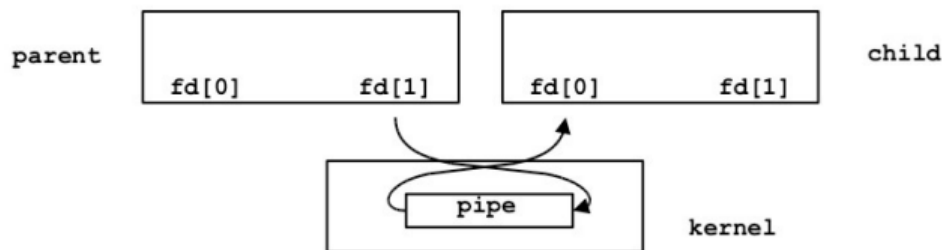
- `fd[1]` è il descrittore di scrittura.

L'output di `fd[1]` è l'input di `fd[0]`. Restituisce 0 in caso di successo, -1 in caso di errore.



Generata da un singolo processo la pipe ha poco significato perché esistono metodi più semplici per comunicare con se stesso.

Tipicamente una pipe viene creata da un processo padre che la usa per comunicare con un processo figlio.



Utilizzo

Una volta creati i descrittori di lettura e scrittura, la pipe può essere usata come un file.

- `read(fd[0], buffer, size)` legge fino a `size` byte dalla pipe e li memorizza in `buffer`.
- `write(fd[1], buffer, size)` scrive fino a `size` byte dalla pipe.
- `close(fd[0])` chiude il descrittore di lettura.
- `close(fd[1])` chiude il descrittore di scrittura.

Una pipe con una estremità chiusa è detta *broken*.

read

Read rimuove i dati dalla pipe: non si possono leggere due volte gli stessi dati, nè si può chiamare `lseek`. (FIFO). Se l'estremo di write è aperto:

- ottiene i dati disponibili, ritornando il numero di byte letti.
- successive chiamate si bloccano fino a quando nuovi dati non saranno disponibili.

Se l'estremo di write è chiuso:

- ottiene i dati disponibili, ritornando il numero di byte letti.
- successive chiamate ritornano 0 per indicare la fine del file.

write

Write aggiunge dati alla pipe.

Se l'estremo di read è aperto:

- scrive i dati, ritornando il numero di byte scritti.
- successive chiamate si bloccano fino a quando non ci sarà spazio disponibile.

Se l'estremo di read è chiuso:

- ritorna un errore **SIGPIPE**.
ignorato/catturato: ritorna -1 con **errno** impostato a **EPIPE**.
default: termina il processo.

Esempio di utilizzo

Il processo parent comunica al processo figlio una stringa, e questi provvede a stamparla.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#define MAXLINE 64

int main(){
    int fd[2];
    pid_t pid;
    char line[MAXLINE];

    if(pipe(fd)<0) perror("pipe error"), exit(1);
    if((pid=fork())<0) perror("fork error"), exit(1);
    else if(pid>0){ //parent
        close(fd[0]); //chiudo il descrittore di lettura
        write(fd[1], "Hello world\n", 12);
    }else{ //child
        close(fd[1]); //chiudo il descrittore di scrittura
        int n = read(fd[0], line, MAXLINE);
        write(STDOUT_FILENO, line, n);
    }
    exit(0);
}
```

9.4 Pipe tra due programmi

Per associare lo `stdout` o lo `stdin` di un programma, rispettivamente, al lato in scrittura o a quello in lettura di una pipe, si può utilizzare la seguente funzione:

```
#include <unistd.h>
int dup2(int oldfd, int newfd);
```

Duplica il descrittore di file `oldfd` su `newfd`.

- `oldfd` è il descrittore di file da duplicare.
- `newfd` è il descrittore di file da sovrascrivere.
- Restituisce `newfd` in caso di successo, -1 in caso di errore.

Se `newfd` è già aperto, viene prima chiuso.

Esempio di utilizzo

Implementa la pipe "who — wc" utilizzando la funzione `dup2`.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <errno.h>
#include <sys/types.h>

int main(){
    int mypipe[2];
    pid_t pid1, pid2;
    if(pipe(mypipe)<0) perror("pipe error"), exit(1);
    if((pid1=fork())<0) perror("fork error"), exit(1);
    else if(pid1==0){ //child 1
        close(mypipe[1]); //chiudo il descrittore di lettura
        if(dup2(mypipe[0], STDIN_FILENO)!=STDIN_FILENO) perror("dup2 error"), exit(1);
        if(execlp("/usr/bin/wc", "wc", NULL)<0) perror("execlp error"), exit(1);
    }else{
        if((pid2=fork())<0) perror("fork error"), exit(2);
        else if(pid2==0){ //child 2
            close(mypipe[0]); //chiudo il descrittore di scrittura
            if(dup2(mypipe[1], STDOUT_FILENO)!=STDOUT_FILENO) perror("dup2 error"), exit(1);
        }
    }
}
```

```

        if(execlp("/usr/bin/who", "who", NULL)<0) perror("execlp error"), exit(1);
    }else{ //parent
        close(mypipe[0]); //chiudo il descrittore di lettura
        close(mypipe[1]); //chiudo il descrittore di scrittura
        waitpid(pid1, NULL, 0);
        waitpid(pid2, NULL, 0);
        exit(0);
    }
}
}

```

9.5 popen e pclose

L'esecuzione di un comando da parte di un processo in modo che quest'ultimo possa riceverne l'output o inviargli l'input è una operazione molto comune, per la quale è dunque utile disporre di una funzione di libreria che ne semplifichi l'implementazione.

La libreria standard di IO fornisce le funzioni **popen** e **pclose** che consentono al programmatore di evitare le azioni esplicite di creazione di una pipe, generazione di un processo figlio e chiusura dei descrittori di file non utilizzati, `exec` di una shell per l'esecuzione del comando e, infine attesa della terminazione di quest'ultimo. Più precisamente, la funzione **popen** crea una pipe, genera un processo figlio, chiude i descrittori di file non utilizzati ed esegue il comando.

La funzione **pclose** attende la terminazione del comando e restituisce il valore di ritorno del comando stesso.

popen

```

#include <stdio.h>
FILE *popen(const char *command, const char *type);

```

- **command** è il comando da eseguire.
- **type** è la modalità di apertura della pipe.
 "r": apre la pipe in lettura.
 "w": apre la pipe in scrittura.
- Restituisce un puntatore a **FILE** in caso di successo, **NULL** in caso di errore.

pclose

```

#include <stdio.h>
int pclose(FILE *stream);

```

- **stream** è il puntatore a **FILE** restituito da **popen**.
- Restituisce il valore di ritorno del comando in caso di successo, -1 in caso di errore.

Esempio di utilizzo

*Implementa la pipe "who — wc" utilizzando le funzioni **popen** e **pclose**.*

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <errno.h>
#include <sys/types.h>

int main(){
    FILE *fp;
    char buf[1024];
    if((fp=popen("who | wc", "r"))==NULL) perror("popen error"), exit(1);
    while(fgets(buf, 1024, fp)!=NULL) printf("%s", buf);
    if(pclose(fp)<0) perror("pclose error"), exit(1);
    exit(0);
}

```

9.6 Pipe con nome (o FIFO)

Una pipe con nome è un file speciale che permette la comunicazione tra processi non imparentati. Devono condividere il "nome" della FIFO.

Creazione di una FIFO

```
#include <sys/types.h>
#include <sys/stat.h>
int mkfifo(const char *pathname, mode_t mode);
```

- `pathname` è il nome della FIFO.
- `mode` è la modalità di apertura della FIFO.
- Restituisce 0 in caso di successo, -1 in caso di errore.

Utilizzo

Una volta creata la FIFO, può essere usata come un file.

- `open(pathname, O_RDONLY)` apre la FIFO in lettura.
- `open(pathname, O_WRONLY)` apre la FIFO in scrittura.
- `close(fd)` chiude il descrittore di file.
- Non è possibile usare `lseek`.
- `unlink(pathname)` rimuove la FIFO.

O_NONBLOCK

È possibile aprire una FIFO in modalità non bloccante, in modo che le chiamate `read` e `write` non si blocchino. Nel contesto dell'apertura di una FIFO, bloccante significa che le chiamate di lettura e scrittura attendono finché non ci sono dati disponibili da leggere o spazio disponibile da scrivere, mentre non bloccante significa che le chiamate di lettura e scrittura restituiscono immediatamente, anche se non ci sono dati da leggere o spazio da scrivere.

PIPE_BUF

`PIPE_BUF` è una costante che rappresenta la dimensione massima di un blocco atomico di dati che possono essere scritti.

Se `PIPE_BUF` è 512 (Di default è 4096), allora una chiamata `write` di 513 byte può scrivere 512 byte in modo atomico, ma il byte rimanente non è garantito che sia scritto in modo atomico, e ciò significa che un'altra chiamata `read` potrebbe leggere i dati in modo parziale.

Esempio di utilizzo

Il processo A comunica al processo B una stringa utilizzando una FIFO, e questi provvede a stamparla.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#define MAXLINE 64

int main(){
    int fd;
    pid_t pid;
    char line[MAXLINE];
```



```

if(mkfifo("fifo", 0666)<0) perror("mkfifo error"), exit(1);
if((pid=fork())<0) perror("fork error"), exit(1);
else if(pid>0){ //parent
    if((fd=open("fifo", O_WRONLY))<0) perror("open error"), exit(1);
    write(fd, "Hello world\n", 12);
}else{ //child
    if((fd=open("fifo", O_RDONLY))<0) perror("open error"), exit(1);
    int n = read(fd, line, MAXLINE);
    write(STDOUT_FILENO, line, n);
}
exit(0);
}

```

Thread

I thread sono "unità esecutive" indipendenti all'interno di un processo, caratterizzate da:

- Un **thread id**
- Un **Insieme di registri**, incluso un contatore di programma ed un puntatore a pila (stack pointer)
- Una **pila** per le variabili locali e gli indirizzi di ritorno (di chiamate a funzioni)
- Una **maschera dei segnali**
- Una **priorità di esecuzione**

Tutti i thread relativi ad uno stesso processo ne condividono:

- Lo **spazio degli indirizzi**
- I **file aperti**
- I **segnali pendenti**
- Le **credenziali**

Il context switch tra thread è più veloce rispetto a quello tra processi, in quanto non è necessario cambiare lo spazio degli indirizzi.

10.1 Utilizzo dei thread

10.1.1 Creazione dei thread

Analogamente ai processi, quando un thread viene creato, esso è associato ad un pezzo di codice; Tuttavia solo se il processo ospite (e quindi il suo corrispondente programma) è single-threaded, il thread è associato all'intero programma.

In caso contrario, il thread è associato ad una funzione del programma, detta **funzione di start** o **funzione di inizio**.

Anche i thread, come i processi, possono assumere gli stati di **running**, **ready**, **blocked** e **terminated**.

Quando un thread termina, la situazione è analoga al caso dei processi: è il programmatore che deve preoccuparsi affinché tutte le risorse impiegate dal thread vengano rilasciate al sistema, facendo in modo che venga effettuata una **wait** a livello di thread.

Quando un programma viene mandato in esecuzione tramite una chiamata `exec`, viene creato un singolo thread, detto **thread principale** o **main thread**.

Ulteriori thread vanno creati esplicitamente.

All'atto della creazione di un thread, è necessario specificare la funzione di avvio: il thread inizierà la propria esecuzione richiamando tale funzione.

La funzione `pthread_create`

La funzione `pthread_create` è utilizzata per creare un nuovo thread.

Il suo prototipo è il seguente:

```
#include <pthread.h>
int pthread_create(pthread_t *tid, const pthread_attr_t *attr, void *(*start_func) (void
*), void *arg);
```

Se la chiamata ha successo, il nuovo thread viene creato e il suo identificatore viene memorizzato nella variabile **tid**.

Il parametro **attr** è un puntatore ad una struttura di tipo **pthread_attr_t**, che contiene gli attributi del thread.

Il parametro **start_func** è un puntatore ad una funzione che viene eseguita dal thread.

Il parametro **arg** è un puntatore ad un oggetto che viene passato alla funzione **start_func** come argomento.

La funzione restituisce 0 in caso di successo, altrimenti restituisce un codice di errore (intero positivo secondo le convenzioni di `<sys/errno.h>`).

10.1.2 Terminazione dei thread

La terminazione di un thread può avvenire in tre diverse circostanze:

- Quando la funzione di avvio termina
- Quando un thread chiama la funzione **pthread_exit**
- Quando un thread chiamante termina (e quindi termina anche il thread chiamato)

La funzione **pthread_exit**

La funzione **pthread_exit** è utilizzata per terminare un thread.

Il suo prototipo è il seguente:

```
#include <pthread.h>
void pthread_exit(void *retval);
```

Il parametro **retval** è un puntatore ad un oggetto che viene passato al thread chiamante. Questo oggetto non può essere un puntatore ad una variabile locale, in quanto la funzione **pthread_exit** termina immediatamente, e quindi la variabile locale non esiste più.

La funzione non restituisce alcun valore.

La funzione **pthread_cancel**

La funzione **pthread_cancel** è utilizzata per terminare un thread.

Il suo prototipo è il seguente:

```
#include <pthread.h>
int pthread_cancel(pthread_t tid);
```

Il parametro **tid** è l'identificatore del thread da terminare.

La funzione restituisce 0 in caso di successo, altrimenti restituisce un codice di errore (intero positivo secondo le convenzioni di `<sys/errno.h>`).

10.1.3 Joining dei thread

La funzione **pthread_join** è utilizzata per attendere la terminazione di un thread.

Sostituisce la funzione **waitpid** per i processi.

Il suo prototipo è il seguente:

```
#include <pthread.h>
int pthread_join(pthread_t tid, void **retval);
```

Il parametro **tid** è l'identificatore del thread da attendere.

Il parametro **retval** è un puntatore ad un oggetto che viene passato al thread chiamante.

Se **retval** è NULL, il thread chiamante non riceve alcun oggetto.

La funzione restituisce 0 in caso di successo, altrimenti restituisce un codice di errore (intero positivo secondo le convenzioni di `<sys/errno.h>`).

10.1.4 Separazione dei thread

La funzione **pthread_detach** è utilizzata per separare un thread, consentendo al sistema di liberare automaticamente le risorse associate al thread una volta che termina la sua esecuzione. Il suo prototipo è il seguente:

```
#include <pthread.h>
int pthread_detach(pthread_t tid);
```

Il parametro **tid** è l'identificatore del thread da separare. La funzione restituisce 0 in caso di successo, altrimenti restituisce un codice di errore (intero positivo secondo le convenzioni di `<sys/errno.h>`). Il suo prototipo è il seguente:

```
#include <pthread.h>
int pthread_detach(pthread_t tid);
```

Il parametro **tid** è l'identificatore del thread da separare.

La funzione restituisce 0 in caso di successo, altrimenti restituisce un codice di errore (intero positivo secondo le convenzioni di `<sys/errno.h>`).

Non è possibile utilizzare la funzione **pthread_join** su un thread separato.

10.2 Esempio di utilizzo dei thread

Il programma stampa i TID del main thread e di due thread creati.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

//Funzione di start dei thread
void *thread_func(void *arg) {
    //Stampa le informazioni del thread
    printf("%s", (char *) arg);
    printf("Thread %d\n", pthread_self());
}

int main() {
    int en;
    pthread_t tid1, tid2;
    char *msg1 = "Hello from thread 1";
    char *msg2 = "Hello from thread 2";

    //Stampa le informazioni del main thread
    printf("Hello from main thread");
    printf("Main thread ID: %d\n", pthread_self());

    //Crea due thread
    if(pthread_create(&tid1, NULL, thread_func, (void *) msg1) != 0) perror("pthread_create"), exit(1);
    if(pthread_create(&tid2, NULL, thread_func, (void *) msg2) != 0) perror("pthread_create"), exit(2);

    //Attende la terminazione dei thread
    if((en = pthread_join(tid1, NULL)) != 0) perror("pthread_join"), exit(1);
    if((en = pthread_join(tid2, NULL)) != 0) perror("pthread_join"), exit(2);

    return 0;
}
```

10.3 Condivisione della memoria

I due thread condividono lo stesso spazio di indirizzamento, e quindi vedono le stesse variabili: se uno dei due modifica una variabile, la modifica è visibile anche all'altro.

Nel caso dei processi tradizionali, una cosa simile è ottenibile solo usando esplicitamente un segmento di memoria condivisa.

10.3.1 Variabili globali

Ma i thread di un processo condividono anche le variabili globali?

La risposta è sì, ma solo se le variabili globali sono **thread-safe**.

Una variabile globale è thread-safe se il suo valore è consistente quando viene letta da più thread contemporaneamente e se il suo valore non viene alterato da un thread mentre un altro thread la sta leggendo.

10.3.2 Variabili locali

Ma i thread di un processo condividono anche le variabili locali?

La risposta è no, in quanto ogni thread ha la propria pila.

Esempio di utilizzo di variabili globali e locali

Il programma crea due thread, che incrementano una variabile globale e una variabile locale.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

//Variabile globale
int global_var = 0;

//Funzione di start dei thread
void *thread_func(void *arg) {
    //Variabile locale
    int local_var = 0;

    //Incrementa le variabili
    global_var++;
    local_var++;

    //Stampa le informazioni del thread
    printf("Thread %lx\n", pthread_self());
    printf("Global var: %d\n", global_var);
    printf("Local var: %d\n", local_var);
}

int main() {
    int en;
    pthread_t tid1, tid2;

    //Crea due thread
    if(pthread_create(&tid1, NULL, thread_func, NULL) != 0) perror("pthread_create"), exit(1);
    if(pthread_create(&tid2, NULL, thread_func, NULL) != 0) perror("pthread_create"), exit(2);

    //Attende la terminazione dei thread
    if((en = pthread_join(tid1, NULL)) != 0) perror("pthread_join"), exit(1);
    if((en = pthread_join(tid2, NULL)) != 0) perror("pthread_join"), exit(2);

    return 0;
}
```

10.4 Segnali e fork

Segnali

I thread di un processo condividono lo stesso insieme di segnali pendenti.

Se arriva un segnale ad un processo, succede che:

- Se il processo ha impostato un handler per il segnale, il gestore viene eseguito da un solo thread
- Se il processo non ha impostato un handler per il segnale e la sua azione predefinita è la terminazione, il processo viene terminato e tutti i thread vengono terminati

Inviare un segnale ad un thread

La funzione **pthread_kill** è utilizzata per inviare un segnale ad un thread.

Il suo prototipo è il seguente:

```
#include <pthread.h>
int pthread_kill(pthread_t tid, int sig);
```

Il parametro **tid** è l'identificatore del thread a cui inviare il segnale.

Il parametro **sig** è il numero del segnale da inviare.

La funzione restituisce 0 in caso di successo, altrimenti restituisce un codice di errore.

Esempio di utilizzo di segnali

Il programma crea due thread, che attendono l'arrivo di un segnale.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <signal.h>

//Funzione di start dei thread
void *thread_func(void *arg) {
    //Stampa le informazioni del thread
    printf("Thread %lx\n", pthread_self());

    //Attende l'arrivo di un segnale
    pause();
}

int main() {
    int en;
    pthread_t tid1, tid2;

    //Crea due thread
    if(pthread_create(&tid1, NULL, thread_func, NULL) != 0) perror("pthread_create"), exit(1);
    if(pthread_create(&tid2, NULL, thread_func, NULL) != 0) perror("pthread_create"), exit(2);

    //Invia un segnale ad entrambi i thread
    if((en = pthread_kill(tid1, SIGUSR1)) != 0) perror("pthread_kill"), exit(1);
    if((en = pthread_kill(tid2, SIGUSR2)) != 0) perror("pthread_kill"), exit(2);

    //Attende la terminazione dei thread
    if((en = pthread_join(tid1, NULL)) != 0) perror("pthread_join"), exit(1);
    if((en = pthread_join(tid2, NULL)) != 0) perror("pthread_join"), exit(2);

    return 0;
}
```

Fork

Quando un processo esegue una fork, il nuovo processo eredita tutti i thread del processo padre.

Quando un thread esegue una fork, il nuovo processo eredita solo il thread che ha eseguito la fork.

10.5 Attributi di un thread

pthread_attr_t

Un thread può essere creato con attributi specifici, che possono essere inizializzati tramite la funzione **pthread_attr_init**:

```
#include <pthread.h>
int pthread_attr_init(pthread_attr_t *attr);
```

Il parametro **attr** è un puntatore ad una struttura di tipo **pthread_attr_t**, che contiene gli attributi del thread.

La funzione restituisce 0 in caso di successo, altrimenti restituisce un codice di errore.

pthread_attr_destroy

Gli attributi di un thread possono essere distrutti tramite la funzione **pthread_attr_destroy**:

```
#include <pthread.h>
int pthread_attr_destroy(pthread_attr_t *attr);
```

Il parametro **attr** è un puntatore ad una struttura di tipo **pthread_attr_t**, che contiene gli attributi del thread. La funzione restituisce 0 in caso di successo, altrimenti restituisce un codice di errore.

detachstate

Un thread può essere creato in "modalità detached" o in "modalità joinable":

- Un thread in modalità detached non può essere atteso ad un altro thread tramite la funzione **pthread_join**
- Un thread in modalità joinable può essere atteso ad un altro thread tramite la funzione **pthread_join**

Il valore di default è la modalità joinable.

Il valore di default può essere modificato tramite la funzione **pthread_attr_setdetachstate**:

```
#include <pthread.h>
int pthread_attr_setdetachstate(pthread_attr_t *attr, int detachstate);
```

Il parametro **attr** è un puntatore ad una struttura di tipo **pthread_attr_t**, che contiene gli attributi del thread. Il parametro **detachstate** può assumere i seguenti valori:

- **PTHREAD_CREATE_JOINABLE**: il thread viene creato in modalità joinable
- **PTHREAD_CREATE_DETACHED**: il thread viene creato in modalità detached

cancelstate

Un thread può bloccare i tentativi di essere cancellato tramite la funzione **pthread_setcancelstate**:

```
#include <pthread.h>
int pthread_setcancelstate(int state, int *oldstate);
```

Il parametro **state** può assumere i seguenti valori:

- **PTHREAD_CANCEL_ENABLE**: il thread può essere cancellato
- **PTHREAD_CANCEL_DISABLE**: il thread non può essere cancellato

Il parametro **oldstate** è un puntatore ad un intero che contiene lo stato precedente.

Il valore di default è **PTHREAD_CANCEL_ENABLE**.

sigmask

Un thread può bloccare i segnali tramite la funzione **pthread_sigmask** la cui sintassi è la seguente:

```
#include <pthread.h>
int pthread_sigmask(int how, const sigset_t *set, sigset_t *oldset);
```

Il parametro **how** può assumere i seguenti valori:

- **SIG_BLOCK**: aggiunge i segnali specificati da **set** all'insieme dei segnali bloccati
- **SIG_UNBLOCK**: rimuove i segnali specificati da **set** dall'insieme dei segnali bloccati
- **SIG_SETMASK**: sostituisce l'insieme dei segnali bloccati con l'insieme specificato da **set**

Il parametro **set** è un puntatore ad un oggetto di tipo **sigset_t** che contiene l'insieme dei segnali da bloccare.

Il parametro **oldset** è un puntatore ad un oggetto di tipo **sigset_t** che contiene l'insieme dei segnali bloccati prima dell'invocazione della funzione.

Concorrenza

11.1 Introduzione

I thread condividono lo spazio degli indirizzi, i file aperti, i semafori, i segnali, ecc.

La concorrenza è un problema di sincronizzazione di accesso alle risorse condivise (Esempio lettura/scrittura di stesse variabili).

Sono necessari meccanismi di sincronizzazione per evitare inconsistenze e deadlock (mutex, condition variable, semafori, monitor, ecc.).

È importante sapere che tutte le operazioni possono non essere atomiche. Per esempio "a = a + 1" non è atomico, ma è composto da 3 operazioni: leggi a, incrementa a, scrivi a.

Esempio

```
#include <stdio.h>
#include <pthread.h>

typedef struct {
    int a;
    int b;
} struttura;
struttura s; // variabile globale

void* incrementa() {
    s.a++;
    s.b++;
    printf("tid=%ld, a=%d, b=%d\n", pthread_self(), s.a, s.b);
    pthread_exit((void*)&s);
}

int main() {
    pthread_t tid[10];
    int i;
    for (i = 0; i < 10; i++) {
        pthread_create(&tid[i], NULL, incrementa, NULL); // thread concorrenti
    }
    for (i = 0; i < 10; i++) pthread_join(tid[i], NULL);
}
```

Esecuzione

```
# user@terminal: gcc -o concorrenza concorrenza.c -lpthread
# user@terminal: ./concorrenza
tid = 139700196177472, a = 1, b = 1
tid = 139700187784768, a = 2, b = 2
tid = 139700137297472, a = 3, b = 3
tid = 139700154082880, a = 7, b = 7
tid = 139700170868288, a = 4, b = 4
tid = 139700128904768, a = 8, b = 8
tid = 139700145690176, a = 10, b = 10
tid = 139700179260992, a = 5, b = 5
tid = 139700162475584, a = 9, b = 9
tid = 139700052424256, a = 6, b = 6
```


Tipi di sincronizzazione

La sincronizzazione può essere di due tipi:

- Per sezione critica
 - Solo quando una struttura viene modificata in un unico punto del codice
 - È sufficiente associare un mutex alla sezione critica
- Per struttura
 - Quando una struttura viene modificata in più punti del codice
 - Utile se più strutture devono essere condivise contemporaneamente
 - È necessario associare un mutex alla struttura

11.2 Mutex Posix

Un mutex Posix è caratterizzato dalle seguenti proprietà:

- è una variabile di tipo `pthread_mutex_t` che può essere inizializzata con diversi attributi (tipo, scopo, etc.)
- può assumere solo due stati alternativi: `locked` (chiuso) e `unlocked` (aperto)
 - `locked`: il thread che ha acquisito il lock può proseguire
 - `unlocked`: il thread che ha rilasciato il lock può proseguire
- può essere chiuso da un solo thread alla volta, gli altri thread che provano ad acquisirlo sono bloccati. Quando il thread che ha acquisito il lock lo rilascia, uno dei thread bloccati può acquisirlo.
- può essere aperto solo dal thread che lo ha chiuso
- deve essere condiviso tra tutti i thread che intendono sincronizzare l'accesso ad una risorsa condivisa (chiamata sezione critica)

11.2.1 inizializzare e distruggere un mutex

```
int pthread_mutex_init(pthread_mutex_t *mutex, const pthread_mutexattr_t *attr);
int pthread_mutex_destroy(pthread_mutex_t *mutex);
```

La funzione `pthread_mutex_init` inizializza il mutex `mutex` con gli attributi `attr`. Se `attr` è NULL, il mutex viene inizializzato con gli attributi di default. La funzione restituisce 0 in caso di successo, altrimenti un codice di errore.

La funzione `pthread_mutex_destroy` distrugge il mutex `mutex`. La funzione restituisce 0 in caso di successo, altrimenti un codice di errore.

Usare i mutex

```
int pthread_mutex_lock(pthread_mutex_t *mutex);
int pthread_mutex_trylock(pthread_mutex_t *mutex);
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

La funzione `pthread_mutex_lock` chiude il mutex `mutex`. Se il mutex è già chiuso, il thread viene bloccato fino a quando il mutex non viene aperto. La funzione restituisce 0 in caso di successo, altrimenti un codice di errore. La funzione `pthread_mutex_trylock` chiude il mutex `mutex`. Se il mutex è già chiuso, la funzione restituisce immediatamente un codice di errore. La funzione restituisce 0 in caso di successo, altrimenti un codice di errore. La funzione `pthread_mutex_unlock` apre il mutex `mutex`. La funzione restituisce 0 in caso di successo, altrimenti un codice di errore.

Esempio

```
#include <stdio.h>
#include <pthread.h>

typedef struct {
    int a;
    int b;
} struttura;

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
struttura s;

void* incrementa() {
    pthread_mutex_lock(&mutex);
    s.a++;
    s.b++;
    printf("tid=%ld, a=%d, b=%d\n", pthread_self(), s.a, s.b);
    pthread_mutex_unlock(&mutex);
    pthread_exit((void*)&s);
}

int main() {
    pthread_t tid[10];
    int i;
    for (i = 0; i < 10; i++) {
        pthread_create(&tid[i], NULL, incrementa, NULL);
    }
    for (i = 0; i < 10; i++) pthread_join(tid[i], NULL);
}
```

Esecuzione

```
# user@terminal: gcc -o mutex mutex.c -lpthread
# user@terminal: ./mutex
tid = 139700196177472, a = 1, b = 1
tid = 139700187784768, a = 2, b = 2
tid = 139700137297472, a = 3, b = 3
tid = 139700154082880, a = 4, b = 4
tid = 139700170868288, a = 5, b = 5
tid = 139700128904768, a = 6, b = 6
tid = 139700145690176, a = 7, b = 7
tid = 139700179260992, a = 8, b = 8
tid = 139700162475584, a = 9, b = 9
tid = 139700052424256, a = 10, b = 10
```

Attributi dei mutex

```
int pthread_mutexattr_init(pthread_mutexattr_t *attr);
int pthread_mutexattr_destroy(pthread_mutexattr_t *attr);
```

Attualmente LinuxThreads supporta solo gli attributi relativi al tipo di mutex.

Tipologie di mutex

- PTHREAD_MUTEX_NORMAL (default)
 - anche detto fast mutex
 - mutex non ricorsivo
 - un thread che chiude un mutex già chiuso da un altro thread causa uno stallo
- PTHREAD_MUTEX_ERRORCHECK
 - mutex non ricorsivo
 - controlla se il mutex è già chiuso da un altro thread e in caso di errore restituisce un codice di errore

- PTHREAD_MUTEX_RECURSIVE

mutex ricorsivo

conta il numero di volte che un thread blocca il mutex e lo rilascia solo se esegue un pari numero di unlock

```
int pthread_mutexattr_settype(pthread_mutexattr_t *attr, int type);
int pthread_mutexattr_gettype(pthread_mutexattr_t *attr, int *type);
```

La funzione *pthread_mutexattr_settype* imposta il tipo di mutex *type* negli attributi *attr*. La funzione restituisce 0 in caso di successo, altrimenti un codice di errore.

La funzione *pthread_mutexattr_gettype* restituisce il tipo di mutex *type* negli attributi *attr*. La funzione restituisce 0 in caso di successo, altrimenti un codice di errore.

Deadlock

Un deadlock è una situazione in cui due o più thread sono bloccati in attesa di una risorsa che non verrà mai rilasciata (Per esempio in una condizione di attesa ciclica).

Un deadlock può essere evitato se si rispettano le seguenti regole:

- ogni thread deve acquisire i lock sempre nello stesso ordine
- ogni thread deve rilasciare i lock sempre nello stesso ordine

Ciò non è sempre possibile!

11.3 Condition Variable

Una variabile di condizione, o condition variable in inglese, è un meccanismo di sincronizzazione spesso utilizzato in combinazione con i mutex per garantire l'accesso sicuro e sincronizzato alle risorse condivise tra i thread.

Una variabile di condizione è associata a un mutex e viene utilizzata per segnalare e attendere eventi specifici. Quando un thread si trova in una situazione in cui deve attendere un certo evento, può chiamare la funzione di attesa sulla variabile di condizione. Questa operazione rilascia il mutex associato e sospende il thread fino a quando un altro thread non segnala l'evento chiamando la funzione di segnalazione sulla stessa variabile di condizione.

L'utilizzo delle variabili di condizione è particolarmente utile quando si desidera sincronizzare l'esecuzione di più thread in base a determinate condizioni o eventi. Ad esempio, un thread potrebbe attendere che una risorsa sia disponibile prima di poterla utilizzare, o potrebbe attendere che una certa condizione sia soddisfatta prima di poter procedere.

Utilizzo

```
int pthread_cond_init(pthread_cond_t *cond, const pthread_condattr_t *attr);
int pthread_cond_destroy(pthread_cond_t *cond);
```

La funzione *pthread_cond_init* inizializza la variabile di condizione *cond* con gli attributi *attr*. Se *attr* è NULL, la variabile di condizione viene inizializzata con gli attributi di default. La funzione restituisce 0 in caso di successo, altrimenti un codice di errore.

La funzione *pthread_cond_destroy* distrugge la variabile di condizione *cond*. La funzione restituisce 0 in caso di successo, altrimenti un codice di errore.

```
int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex);
```

Questa funzione blocca il thread corrente fino a quando un altro thread segnala la variabile di condizione *cond*. Il mutex *mutex* viene rilasciato automaticamente mentre il thread è bloccato e ri-acquisito prima che la funzione ritorni.

```
pthread_cond_timedwait(pthread_cond_t *cond, pthread_mutex_t *mutex, const struct timespec *
    abstime)
```

Questa funzione è simile a *pthread_cond_wait*, ma ha un timeout. Se nessun altro thread ha segnalato la variabile di condizione *cond* entro il tempo specificato in *abstime*, la funzione ritorna.

```
pthread_cond_signal(pthread_cond_t *cond)
```

Questa funzione segnala la variabile di condizione cond. Se ci sono thread bloccati su cond, almeno uno di questi thread viene sbloccato.

```
pthread_cond_broadcast(pthread_cond_t *cond)
```

Questa funzione è simile a pthread_cond_signal, ma sblocca tutti i thread che sono attualmente bloccati su cond.

11.4 Thread specific data (TSD)

Complementarietà tra thread e processi

- Thread:
 - Semplice scambiare dati tra thread
 - Appositi meccanismi per avere dati privati per ogni thread (Thread specific data)
- Processi
 - Dati privati per ogni processo
 - Appositi meccanismi per scambiare dati tra processi (Memoria condivisa, Pipe, Socket, ecc.)

Ogni thread possiede un'area di memoria privata, la TSD area, indicizzata da chiavi (interi non negativi). La TSD area contiene associazioni tra le chiavi ed un valore di tipo *void**. Diversi thread possono usare le stesse chiavi ma i valori associati variano di thread in thread.

Inizialmente tutte le chiavi sono associate a *NULL*.

Utilizzo

```
int pthread_key_create(pthread_key_t *key, void (*destructor)(void*));  
int pthread_key_delete(pthread_key_t key);
```

La funzione *pthread_key_create* crea una nuova chiave thread-specifica e la memorizza in key. La funzione restituisce 0 in caso di successo, altrimenti un codice di errore.

La funzione *pthread_key_delete* elimina la chiave thread-specifica key. La funzione restituisce 0 in caso di successo, altrimenti un codice di errore.

```
int pthread_setspecific(pthread_key_t key, const void *value);  
void *pthread_getspecific(pthread_key_t key);
```

La funzione *pthread_setspecific* imposta il valore della chiave thread-specifica key al valore value. La funzione restituisce 0 in caso di successo, altrimenti un codice di errore.

La funzione *pthread_getspecific* restituisce il valore della chiave thread-specifica key.

Esempio

```
#include <stdio.h>  
#include <pthread.h>  
#include <stdlib.h>  
  
static pthread_key_t key;  
  
void write_to_thread_log (const char* message);  
void close_thread_log (void* thread_log);  
void* thread_function (void* args);  
  
int main() {  
    int i;  
    pthread_t threads[5];  
    pthread_key_create(&key, close_thread_log);  
    for (i = 0; i < 5; i++) pthread_create(&threads[i], NULL, thread_function, NULL);  
    for (i = 0; i < 5; i++) pthread_join(threads[i], NULL);  
    pthread_key_delete(key);  
    exit(EXIT_SUCCESS);  
}
```

```

void write_to_thread_log (const char* message) {
    FILE* thread_log = (FILE*)pthread_getspecific(key);
    fprintf(thread_log, "%s\n", message);
}

void close_thread_log (void* thread_log) {
    fclose((FILE*)thread_log);
}

void* thread_function (void* args) {
    char thread_log_filename[20];
    FILE* thread_log;
    sprintf(thread_log_filename, "thread%d.log", (int)pthread_self());
    thread_log = fopen(thread_log_filename, "w");
    pthread_setspecific(key, thread_log);
    write_to_thread_log("Thread starting...");
    return NULL;
}

```

Esecuzione

```

# user@terminal: gcc -o tsd tsd.c -lpthread
# user@terminal: ./tsd
# user@terminal: ls
tsd.c thread139700196177472.log thread139700187784768.log thread139700137297472.log thread139700154082880.log
thread139700170868288.log tsd
# user@terminal: cat thread139700196177472.log
Thread starting...

```

Socket

12.1 Introduzione

Una socket è un'astrazione software che permette di realizzare una comunicazione tra due processi, che possono essere eseguiti sulla stessa macchina o su macchine diverse, attraverso una rete indipendentemente dai linguaggi di programmazione, dal sistema operativo e dall'hardware utilizzato.

Una socket identificata da un indirizzo IP e da un numero di port ed è un canale di comunicazione bidirezionale, ovvero permette di inviare e ricevere dati.

Domini e Stili di Comunicazione

I socket permettono di specificare il tipo di comunicazione attraverso le nozioni di dominio e di stile.

Il dominio di un socket equivale alla scelta di una famiglia di protocolli. Le correnti relaz. del kernel di molti Unix prevedono ventisei diverse famiglie.

Ogni dominio è individuato univocamente da un intero non negativo, cui corrispondono una o più costanti simboliche del tipo `PF_nomefamiglia`, definite in `<sys/socket.h>`.

I domini più utilizzati sono:

- `PF_LOCAL` o `PF_UNIX`: per la comunicazione tra processi che risiedono sulla stessa macchina;
- `PF_INET`: per la comunicazione tra processi che risiedono su macchine diverse e che utilizzano il protocollo TCP/IP con indirizzi IPv4;
- `PF_INET6`: per la comunicazione tra processi che risiedono su macchine diverse e che utilizzano il protocollo TCP/IP con indirizzi IPv6;
- `PF_APPLETALK`: per la comunicazione tra processi che utilizzano le reti AppleTalk;
- `PF_IPX`: per la comunicazione tra processi che utilizzano le reti Novell IPX;

A ciascun dominio possono corrispondere in teoria uno o più schemi di indirizzamento, ossia tipi di indirizzi, per cui i socket prevedono la nozione di famiglia di indirizzi, ciascuna individuata univocamente attraverso un numero non negativo cui corrisponde (sempre tramite l'header `<sys/socket.h>`) una costante simbolica del tipo `AF_nomefamiglia`.

Stili di Comunicazione

Lo stile di comunicazione è individuato da costanti simboliche del tipo `SOCK_nomeStile`, definite in `<sys/socket.h>`.

I principali stili di comunicazione sono:

- `SOCK_STREAM`: per la comunicazione affidabile, orientata alla connessione, con flusso di byte;
- `SOCK_DGRAM`: per la comunicazione non affidabile, non orientata alla connessione, con messaggi;
- `SOCK_RAW`: per la comunicazione a basso livello ai protocolli di rete e alle varie interfacce;

Assegnato un dominio, la scelta dello stile di comunicazione corrisponde in pratica ad individuare uno specifico protocollo tra quelli appartenenti al dominio.

Non tutte le combinazioni di dominio e stile sono permesse, in quanto non è detto che in una famiglia di protocolli esista un elemento per ciascuno dei possibili stili.

Indirizzamento

In UNIX le strutture dati per la gestione degli indirizzi sono progettate per essere adattate a diversi domini di comunicazione e protocolli. Ad esempio:

- `sa_family = AF_INET` e `sa_data = sin_addr` per il dominio `PF_INET` e il protocollo IP;
- `sa_family = AF_UNIX` e `sa_data = sun_path` per il dominio `PF_UNIX` e il protocollo LOCAL;

Indirizzo generico

```
struct sockaddr {
    unsigned short sa_family;
    char sa_data[14];
};
```

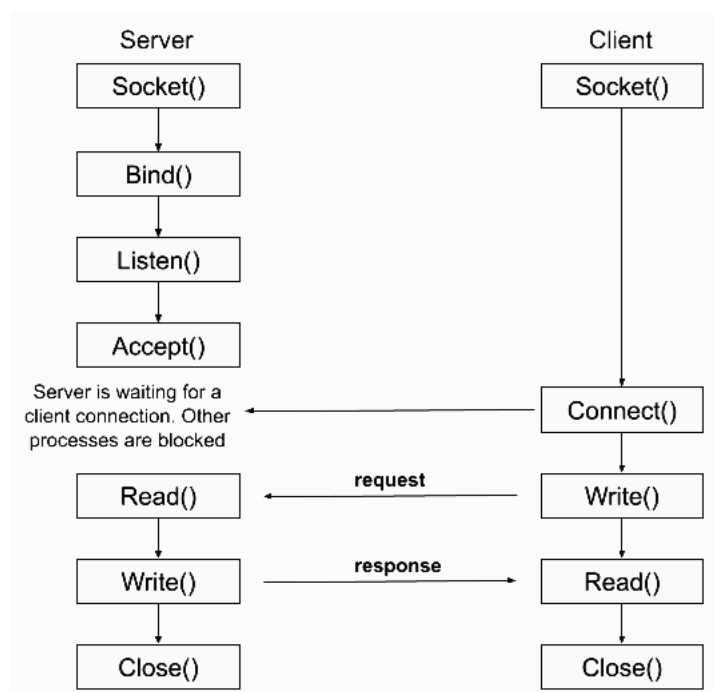
Indirizzo per il dominio AF_INET

Solitamente siamo interessati a `sa_family = AF_INET`, cioè al dominio `PF_INET` e al protocollo IP. In tal caso servono 2 byte per un numero di porta e 4 byte per un indirizzo IP.

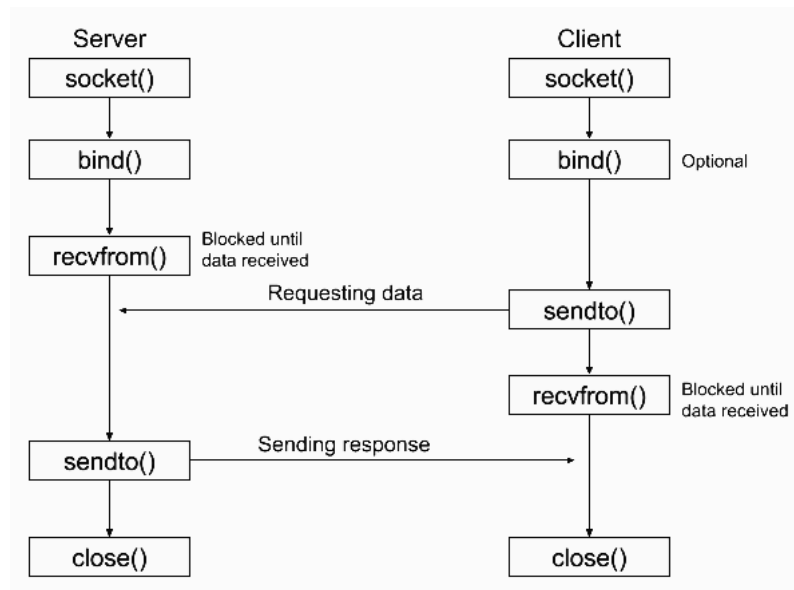
```
#include <netinet/in.h>
struct sockaddr_in {
    unsigned short sin_family; // Address family
    unsigned short sin_port;   // Port number
    struct in_addr sin_addr;    // IP Internet address
    unsigned char sin_zero[8];
};
```

12.2 Socket API

Flusso di comunicazione TCP (SOCK_STREAM)



Flusso di comunicazione UDP (SOCK_DGRAM)



Creazione di un socket

```
#include <sys/types.h>
#include <sys/socket.h>
int socket(int domain, int type, int protocol);
```

- **domain**: dominio di comunicazione;
- **type**: stile di comunicazione;
- **protocol**: protocollo da utilizzare. Se `protocol = 0`, il protocollo viene scelto in base al dominio e allo stile di comunicazione;
- **return**: il file descriptor del socket creato, in caso di successo, -1 in caso di errore;

Assegnazione di un indirizzo ad un socket

```
#include <sys/types.h>
#include <sys/socket.h>
int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
```

- **sockfd**: file descriptor del socket;
- **addr**: puntatore alla struttura dati che contiene l'indirizzo da assegnare al socket;
- **addrlen**: dimensione della struttura dati `addr`;
- **return**: 0 in caso di successo, -1 in caso di errore;

Ascolto su una socket

```
#include <sys/types.h>
#include <sys/socket.h>
int listen(int sockfd, int backlog);
```

`listen` è utilizzata solo per i socket di tipo `SOCK_STREAM`;

- **sockfd**: file descriptor del socket;
- **backlog**: numero massimo di connessioni pendenti che possono essere accodate per il socket;
- **return**: 0 in caso di successo, -1 in caso di errore;

Accettazione di una connessione

```
#include <sys/types.h>
#include <sys/socket.h>
int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);
```

`accept` è utilizzata solo per i socket di tipo `SOCK_STREAM`;

- `sockfd`: file descriptor del socket;
- `addr`: puntatore alla struttura dati che conterrà l'indirizzo del processo che si connette al socket;
- `addrlen`: puntatore alla variabile che conterrà la dimensione della struttura dati `addr`;
- `return`: il file descriptor del socket connesso, in caso di successo, -1 in caso di errore;

Connessione ad un socket

```
#include <sys/types.h>
#include <sys/socket.h>
int connect(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
```

`connect` è utilizzata solo per i socket di tipo `SOCK_STREAM`;

- `sockfd`: file descriptor del socket;
- `addr`: puntatore alla struttura dati che contiene l'indirizzo del processo a cui connettersi;
- `addrlen`: dimensione della struttura dati `addr`;
- `return`: 0 in caso di successo, -1 in caso di errore;

Invio di dati su un socket

`write`

È possibile utilizzare la funzione `write` per inviare dati su un socket come se fosse un file.

```
#include <unistd.h>
ssize_t write(int fd, const void *buf, size_t count);
```

- `fd`: file descriptor del socket;
- `buf`: puntatore al buffer contenente i dati da inviare;
- `count`: numero di byte da inviare;
- `return`: numero di byte inviati, in caso di successo, -1 in caso di errore;

`send`, `sendto`, `sendmsg`

È possibile utilizzare funzioni specifiche per l'invio di dati su un socket.

```
#include <sys/types.h>
#include <sys/socket.h>
ssize_t send(int sockfd, const void *buf, size_t len, int flags);
ssize_t sendto(int sockfd, const void *buf, size_t len, int flags, const struct sockaddr *
    dest_addr, socklen_t addrlen);
ssize_t sendmsg(int sockfd, const struct msghdr *msg, int flags);
```

- `sockfd`: file descriptor del socket;
- `buf`: puntatore al buffer contenente i dati da inviare;
- `len`: numero di byte da inviare;

- **flags**: flag per la funzione;
- **dest_addr**: puntatore alla struttura dati che contiene l'indirizzo del processo a cui inviare i dati;
- **addrlen**: dimensione della struttura dati **dest_addr**;
- **msg**: puntatore alla struttura dati che contiene i dati da inviare;
- **return**: numero di byte inviati, in caso di successo, -1 in caso di errore;

Differenze tra **write**, **send**, **sendto** e **sendmsg**

In questa sezione, discuteremo le differenze tra le funzioni di sistema **write**, **send**, **sendto** e **sendmsg**.

write La funzione **write** è una delle funzioni di I/O più basilari in Unix/Linux. È usata per scrivere dati su un file o un socket. È generalmente usata quando si lavora con file o quando si scrive su un socket in una connessione TCP.

send La funzione **send** è specifica per i socket. È simile a **write**, ma include alcune opzioni aggiuntive, come la possibilità di inviare flag. È generalmente usata quando si ha bisogno di un controllo più fine sulla comunicazione del socket.

sendto La funzione **sendto** è simile a **send**, ma permette di specificare un indirizzo di destinazione. Questo la rende utile per i protocolli senza connessione come UDP, dove l'indirizzo di destinazione potrebbe cambiare da un messaggio all'altro.

sendmsg La funzione **sendmsg** è una versione più avanzata di **send** e **sendto** che permette di inviare più messaggi contemporaneamente e di controllare vari aspetti del messaggio come l'indirizzo di destinazione e le opzioni del protocollo. È generalmente usata quando si ha bisogno di un controllo molto fine sulla comunicazione del socket.

Ricezione di dati da un socket

read

È possibile utilizzare la funzione **read** per ricevere dati da un socket come se fosse un file.

```
#include <unistd.h>
ssize_t read(int fd, void *buf, size_t count);
```

- **fd**: file descriptor del socket;
- **buf**: puntatore al buffer in cui salvare i dati ricevuti;
- **count**: numero di byte da ricevere;
- **return**: numero di byte ricevuti, in caso di successo, -1 in caso di errore;

recv, **recvfrom**, **recvmsg**

È possibile utilizzare funzioni specifiche per la ricezione di dati da un socket.

```
#include <sys/types.h>
#include <sys/socket.h>
ssize_t recv(int sockfd, void *buf, size_t len, int flags);
ssize_t recvfrom(int sockfd, void *buf, size_t len, int flags, struct sockaddr *src_addr,
                 socklen_t *addrlen);
ssize_t recvmsg(int sockfd, struct msghdr *msg, int flags);
```

- **sockfd**: file descriptor del socket;
- **buf**: puntatore al buffer in cui salvare i dati ricevuti;
- **len**: numero di byte da ricevere;

- **flags**: flag per la funzione;
- **src_addr**: puntatore alla struttura dati che conterrà l'indirizzo del processo che ha inviato i dati;
- **addrlen**: puntatore alla variabile che conterrà la dimensione della struttura dati **src_addr**;
- **msg**: puntatore alla struttura dati che conterrà i dati ricevuti;
- **return**: numero di byte ricevuti, in caso di successo, -1 in caso di errore;

Differenze tra `read`, `recv`, `recvfrom` e `recvmsg`

In questa sezione, discuteremo le differenze tra le funzioni di sistema `read`, `recv`, `recvfrom` e `recvmsg`.

read La funzione `read` è una delle funzioni di I/O più basilari in Unix/Linux. È usata per leggere dati da un file o un socket. È generalmente usata quando si lavora con file o quando si legge da un socket in una connessione TCP.

recv La funzione `recv` è specifica per i socket. È simile a `read`, ma include alcune opzioni aggiuntive, come la possibilità di ricevere flag. È generalmente usata quando si ha bisogno di un controllo più fine sulla comunicazione del socket.

recvfrom La funzione `recvfrom` è simile a `recv`, ma permette di specificare un indirizzo di origine. Questo la rende utile per i protocolli senza connessione come UDP, dove l'indirizzo di origine potrebbe cambiare da un messaggio all'altro.

recvmsg La funzione `recvmsg` è una versione più avanzata di `recv` e `recvfrom` che permette di ricevere più messaggi contemporaneamente e di controllare vari aspetti del messaggio come l'indirizzo di origine e le opzioni del protocollo. È generalmente usata quando si ha bisogno di un controllo molto fine sulla comunicazione del socket.

Chiusura di un socket

```
#include <unistd.h>
int close(int fd);
```

- **fd**: file descriptor del socket;
- **return**: 0 in caso di successo, -1 in caso di errore;

12.3 Risoluzione dei nomi

La risoluzione dei nomi è il processo di conversione di un nome di host leggibile dall'uomo nel suo corrispondente indirizzo IP, consentendo la comunicazione tra dispositivi in una rete.

Risoluzione di un nome

La funzione `gethostbyname` viene utilizzata per risolvere un nome di host in un host corrispondente.

```
#include <netdb.h>
struct hostent *gethostbyname(const char *name);
```

- **name**: nome dell'host;
- **return**: puntatore alla struttura dati che contiene l'indirizzo IP dell'host, in caso di successo, NULL in caso di errore;

Risoluzione di un indirizzo IP

La funzione `gethostbyaddr` viene utilizzata per risolvere un indirizzo IP in un host corrispondente.

```
#include <netdb.h>
struct hostent *gethostbyaddr(const void *addr, socklen_t len, int type);
```

- **addr**: puntatore all'indirizzo IP;
- **len**: dimensione dell'indirizzo IP;
- **type**: tipo di indirizzo IP;
- **return**: puntatore alla struttura dati che contiene il nome dell'host, in caso di successo, `NULL` in caso di errore;

Esercizi svolti

13.1 Variabili di Shell

Creare un file che si chiami come l'utente corrente

```
# user@terminal: touch $USER
```

Creare un file che si chiami come l'host corrente, e che contenga il nome dell'host corrente

```
# user@terminal: touch $HOSTNAME.txt
```

```
# user@terminal: touch $USER > $HOSTNAME.txt
```

13.2 grep

Elencare i file con permesso di esecuzione per il proprietario

```
# user@terminal: ls -l | grep "^...x" > elenco.txt
```

Elencare le directory il cui nome inizia per maiuscola

```
# user@terminal: ls -l | grep ^d | grep '<[:upper:]][^ ]*$'
```

O in alternativa

```
# user@terminal: ls -ld */ | grep ^[:upper:]
```

Elencare i file con permesso di esecuzione oppure di scrittura per il gruppo di appartenenza

```
# user@terminal: ls -l | grep "^.....x|^.....w"
```

E per rimuovere le cartelle

```
# user@terminal: ls -l | grep "^.....x|^.....w" | grep -v "^d"
```

13.3 Processi

Stampare il seguente stato interno del processo bash più recente

- PID del processo
- Dimensione della memoria residente
- Tempo di CPU utilizzato
- Stato del processo
- Nome del comando

```
# user@terminal: ps -o 'pid %cpu size time state cmd'
```

Individuate tutti i processi bloccati (Stato S) ed in esecuzione (Stato R) sulla macchina

```
# user@terminal: ps -a -o "state pid" | grep "S\R"
```

O in alternativa:

```
# user@terminal: ps -e -l
```

13.4 Concatenazione e sostituzione comandi

Scrivere un unico comando che permetta di eseguire le seguenti operazioni

- Creare una cartella **EsercitazioneLSO-1** nella directory di lavoro
- Creare un file di testo chiamato **provaFile.txt** che contiene username e HOSTNAME
- Creare una variabile che contiene il contenuto di provaFile
- Aggiungere alla variabile il nome del file
- Creare un file che contiene il contenuto della variabile

```
# user@terminal: mkdir EsercitazioneLSO-1; cd EsercitazioneLSO-1; var=$(cat $(echo -e $(echo $USER $HOSTNAME " " > provaFile.txt)) provaFile.txt$(ls *.txt); echo -e $var >new_file.txt
```

Scrivere un unico comando che permetta di eseguire le seguenti operazioni

- Creare un file che si chiami come l'utente corrente
- Assegnare alla variabile *x* l'elenco dei file che cominciano con un punto
- Scrivere alcune parole nel file nomi.txt. Successivamente, per ogni parola contenuta nel file, creare un file con nome uguale a quella parola

13.5 Word counter

*Assegnare alla variabile *x* il numero di righe di un file a vostra scelta*

```
# user@terminal: x=$(wc -l testo.txt)
```

Contare i file nella directory corrente che contengono una z nel nome

```
# user@terminal: ls | grep z | wc -l
```

Contare i file nella directory corrente che non contengono una z all'inizio del nome

```
# user@terminal: ls | grep ^[~z] | wc -l
```

13.6 Sort

Elencare i file della directory corrente in ordine alfabetico inverso

```
# user@terminal: ls | sort -r
```

Scrivere nel file "elenco" l'elenco dei file nella directory corrente, in ordine alfabetico

```
# user@terminal: ls | sort -o elenco
```

13.7 Head e Tail

Stampare la 3 e la quarta riga del file provaFile.txt

```
# user@terminal: head -4 provaFile.txt | tail -2
```

Stampare le penultime 3 righe del file provaFile.txt

```
# user@terminal: tail -4 provaFile.txt | head -3
```

Stampare l'n-esima riga del file provaFile.txt

Da sostituire n con il numero desiderato # user@terminal: head -n provaFile.txt | tail -1

13.8 IFS

Usando la virgola come delimitatore, creare un file per ogni elemento separato dal delimitatore usato

```
# user@terminal: text='file1,file2,file3'
# user@terminal: IFS=', '
# user@terminal: touch $text
# user@terminal: ls
file1 file2 file3
```

13.9 Script

Esegui le seguenti istruzioni:

1. Con un edito di testi creare un file con il seguente contenuto:

```
#!/bin/bash
echo "Hello world!"
ls
```

2. Salvarlo con il nome "mio_script"

3. Dargli i permessi di esecuzione

4. Eseguirlo digitando "./mio_script"

```
# user@terminal: nano mio_script
Digitazione script...
# user@terminal: chmod u+x mio_script
# user@terminal: ./mio_script
```

Scivere uno script "eccho" che prende un argomento e lo stampa due volte

```
# user@terminal: nano eccho
#!/bin/bash
echo $1 $1
# user@terminal: chmod u+x eccho
# user@terminal: ./eccho prova
prova prova
```

Scivere uno script "bis" che prende un comando come argomento e lo esegue due volte

```
# user@terminal: nano eccho
#!/bin/bash
$*
$*
# user@terminal: chmod u+x bis
# user@terminal: ./bis ls
```

```
-rwxr--r-- 1 proprietario gruppo 1298 Sep 26 15:28 bis
-rw-r--r-- 1 proprietario gruppo 1298 Sep 26 14:28 fil1.txt
-rwxr--r-- 1 proprietario gruppo 1298 Sep 26 15:28 bis
-rw-r--r-- 1 proprietario gruppo 1298 Sep 26 14:28 fil1.txt
```

Fare uno script che cerca di creare un file all'interno della directory /etc ed andare a stampare l'errore

```
# user@terminal: nano test
#!/bin/bash
mkdir /etc/dir
echo $?
# user@terminal: chmod u+x bis
# user@terminal: ./test
mkdir: cannot create directory '/etc/dir': Permission denied
```

1

13.10 Linguaggio bash

Si realizzi uno script "scriviNumero.sh" che scrive a video i numeri da 0 a N.

Il valore N viene passato alla chiamata

```
#!/bin/bash
n=0
while [ ! $n -eq $1 ]
do
    echo $n
    n=$((n+1))
done
```

Il valore N viene letto in runtime

```
#!/bin/bash
n=0
if [ -z $1 ]
then
    echo "Inserisci valore:"
    read c
else
    c=$1
fi
while [ ! $n -eq $c ]
do
    echo $n
    n=$((n+1))
done
```

Si realizzi uno script che chiameremo "scriviNumeri.sh" che scrive a video i numeri da 20 a 10.

```
#!/bin/bash
for x in $(seq 20 -1 10)
do
    echo "x: $x"
done
```

13.11 Il comando sed

Usa grep e sed per selezionare tutte le linee che contengono la parola "When" oppure la parola "when".

```
# user@terminal: grep \[W,w]hen testo.txt
# user@terminal: sed -n '/[W,w]hen/p' testo.txt
```

Nel seguente file di testo, sostituire la formattazione Nome Cognome Città con Sig. Nome-Cognome Da Città.

Davide Pierro Napoli

Emilia Napolano Milano

Luigi Picarella Torino

```
# user@terminal: sed -e 's/\(.*\) \(.*\) \(.*\) /Sig \1-\2 Da \3/' myfile.txt
```

13.12 Il comando awk

*Dato il seguente file dati.txt di testo calcolare il subtotale per ogni cliente: 100*2, 200*8, 500*2 e poi il totale.*

100:2:Cliente 1

200:8:Cliente 2

500:2:Cliente 3

```
# user@terminal: awk -F ':' '{ somma=$1*$2; print $3 ": " $1*$2; totale=totale+somma } END { print "Totale: " totale }' dati.txt
```


Dato il file di testo tabella.txt così fatto:

Luigi
Marco
Giovanni
Luigi
Giorgio
Luca
Maria

Scrivere uno script che eseguito da awk sul file stampa il contenuto del file tutto su una riga

```
# user@terminal: awk -F '\n' 'BEGIN { ORS=" " } { for (i=0; i<NF; i++) print $i }' tabella.txt
```

Dato il seguente file di testo poesia.txt:

La donzelletta vien dalla campagna,
In sul calar del sole,
Col suo fascio dell'erba; e reca in mano
Un mazzolin di rose e di viole,
Onde, siccome suole,
Ornare ella si appresta
Dimani, al dì di festa, il petto e il crine.
Siede con le vicine
Su la scala a filar la vecchierella
Incontro là dove si perde il giorno;
E novellando vien del suo buon tempo,
Quando ai di della festa ella si ornavo,
Ed ancor sana e snella
Solea danzar la sera intra di quei
Ch'ebbe compagni dell'età più bella.

scrivere uno script che eseguito da awk sul file stampa il contenuto del file fino alla stringa "crine"

```
# user@terminal: awk 'print /crine/ exit' poesia.txt
```

Dato il seguente file di testo tabella.txt Luigi;Marco;Nino;Nicola;Alberto;Luigi;

Marco;Giovanni;Giorgio;Maria;
Giovanni;Ottavio;Luigi;Luigi;Nino;
Luigi;Guglielmo;Nino;Ennio;Luigi;
Giorgio; Vittorio;
Luca;Marta;Maria;Luigi;Salvo;
Maria;Nino;Ennio;Luigi;Maria;

scrivere uno script che eseguito da awk sul file determini quante volte è contenuto il nome "Luigi"

```
# user@terminal: awk -n 'BEGIN{ RS=";" } /Luigi/ { luigi=luigi+1 } END{ print "Il nome Luigi appare " luigi " volte" }' tabella.txt
```

13.13 Input e Output

Scrivere un programma che mostra il contenuto di un file a byte alterni (un carattere sì e uno no).

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>
#include <errno.h>
#include <fcntl.h>
#include <sys/stat.h>

int main(){
    int fd;

    if((fd=open("file.txt", O_RDONLY))<0){
        perror("open error");
    }

    int i=0;
    char letto;
```

```

        while(read(fd, &letto, 1)){
            if(i%2==0) { printf("%c", letto); }
            i++;
        }
        close(fd);
        return 0;
}

```

13.14 Processi

Scrivere un programma C che apre un file. effettua una fork e scrive messaggi diversi sul file a seconda che sia padre o figlio.

Successivamente spostare la open dopo la fork.

```

#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/stat.h>

int main(int argc, char* argv[]) {
    //effettuare una fork
    pid_t pid = fork();
    if (pid == -1) {
        perror("fork");
    }
    // aprire un file
    int fd = open("file.txt", O_WRONLY | O_CREAT);
    if (fd == -1) {
        perror("open");
    }
    // scrivere cose diverse se sei padre e se sei figlio
    if (pid == 0) {
        // sono il figlio
        // scrivere la stringa "sono il figlio" nel file
        int nwrite = write(fd, "sono il figlio\n", 15);
        if (nwrite == -1) {
            perror("write");
        }
    } else {
        // sono il padre
        // scrivere la stringa "sono il padre" nel file
        int nwrite = write(fd, "sono il padre\n", 14);
        if (nwrite == -1) {
            perror("write");
        }
    }
    // chiudere il file
    close(fd);
}

```

Programma che genera due processi, il primo scrive una stringa su di un file, il secondo la legge

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>
#include <sys/types.h>
#include <sys/stat.h>

int main(){
    pid_t pid=vfork();
    int fd = open("file.txt", O_CREAT|O_RDWR, S_IRUSR|S_IWUSR); //Esegue la fork
    if(pid<0) perror("vfork error"); //Apri il file
    else if(pid==0){ //Se la fork fallisce
        char buf[]="MESSAGGIO DEL FIGLIO\n"; //Messaggio da scrivere
        printf("Il figlio scrive: %s", buf); //Stampa il messaggio
        write(fd, buf, sizeof(buf)); //Scrive sul file
        exit(0); //Esce dal figlio
    }else{

```

```

        wait(NULL);
        printf("Il padre legge: ");
        char c;
        while(read(fd, &c, 1)) printf("%c", c);
    }
    close(fd);
}
//Aspetta il figlio
//Stampa il messaggio
//Carattere da leggere
//Legge il file
//Chiude il file

```

Scrivere un programma che manda in esecuzione un eseguibile il cui filename e' inserito come argomento sulla linea comando e ne aspetta la terminazione.

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(int argc, char* argv[]){
    if(argv[1]==NULL) perror("Inserire il nome di un programma\n"), exit(1);
    pid_t pid=fork();
    if(pid<0) perror("vfork error"), exit(1);
    else if(pid==0) execlp(argv[1], argv[1], NULL);
    else wait(NULL);
    printf("Il processo %d ha finito\n", pid);
}

```

13.15 Pipe

Scrivere un programma C che crea un figlio

- il padre invia al figlio 10 numeri interi casuali al ritmo di uno al secondo, e poi termina
- il figlio riceve i numeri dal padre e li stampa sul terminale
- il figlio termina dopo aver ricevuto il decimo numero

I due processi comunicano tramite una pipe

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <time.h>
#include <sys/wait.h>

int main(int argc, char **argv){
    int pid;
    int fd[2];
    srand(time(NULL));

    if(pipe(fd)<0) perror("pipe"), exit(1);

    if((pid=fork())<0) perror("fork"), exit(1);

    if(pid == 0){ // figlio
        close(fd[1]); // chiude l'estremo di scrittura
        for(int i = 0; i < 10; i++){
            int buf;
            read(fd[0], &buf, sizeof(int));
            printf("Figlio legge: %d\n", buf);
        }
        close(fd[0]); // chiude l'estremo di lettura
        printf("\nFiglio termina\n");
        exit(0); // termina
    }else{// padre
        close(fd[0]); // chiude l'estremo di lettura
        for(int i = 0; i < 10; i++){
            int n = rand() % 100; // genera un numero casuale tra 0 e 99
            printf("\nGenerato %d' numero\nPadre scrive: %d\n",i+1, n);
            write(fd[1], &n, sizeof(int));
            sleep(1);
        }
        close(fd[1]); // chiude l'estremo di scrittura
        wait(0); // aspetta la terminazione del figlio
    }
}

```

```

        printf("Padre termina\n");
    }
    return 0;
}

```

Scrivere un programma C che crea un figlio

- il padre entra in un ciclo in cui legge da terminale un numero intero x e manda al figlio il numero $x*2$
- il padre esce dal ciclo e termina quando l'utente immette il numero 0
- il figlio riceve i numeri dal padre e li stampa sul terminale
- il figlio termina quando riceve 0 dal padre

I due processi comunicano tramite una pipe

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(int argc, char **argv){
    int pid;
    int fd[2];

    if(pipe(fd)<0) perror("pipe"), exit(1);

    if((pid=fork())<0) perror("fork"), exit(1);

    if(pid == 0){ // figlio
        close(fd[1]);
        int buf = 1;
        while(buf!=0){
            read(fd[0], &buf, sizeof(int));
            printf("Figlio legge: %d\n\n", buf);
        }
        close(fd[0]);
        printf("Figlio termina\n");
        exit(0);
    }else{// padre
        close(fd[0]);
        int n = 1;
        while(n!=0){
            sleep(1);
            printf("Inserisci un numero: ");
            scanf("%d", &n);
            printf("Padre scrive: %d\n", n);
            write(fd[1], &n, sizeof(int));
        }
        close(fd[1]);
        wait(0);
        printf("Padre termina\n");
    }
    return 0;
}

```

13.16 Thread

Realizzare un programma C che accetta da riga di comando due numeri interi n ed m , e crea np produttori ed nc consumatori

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

char *buffer; // Buffer
int N; // Dimensione del buffer
int count = 0; // Contatore degli elementi nel buffer

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER; // Mutex per la mutua esclusione
pthread_cond_t full = PTHREAD_COND_INITIALIZER; // Variabile condizione per il buffer
vuoto

```

```

pthread_cond_t empty = PTHREAD_COND_INITIALIZER; // Variabile condizione per il buffer
pieno

void leggi_buffer(char p) {
    printf("%c buffer: ", p);
    for (int i = 0; i < count; i++) printf("%c ", buffer[i]);
    printf("\n");
}

void* producer(void* arg) {
    char item = 'A'; // Elemento da produrre

    pthread_mutex_lock(&mutex); // Acquisizione del mutex

    // Verifica se il buffer e' pieno
    while (count == N) pthread_cond_wait(&full, &mutex); // Attesa sulla variabile
    condizione full

    // Scrive l'elemento nel buffer
    buffer[count] = item;
    count++;

    leggi_buffer('P');

    pthread_cond_signal(&empty); // Sveglia un consumatore in attesa sulla variabile
    condizione empty
    pthread_mutex_unlock(&mutex); // Rilascio del mutex

    return NULL;
}

void* consumer(void* arg) {
    char item; // Elemento da consumare

    pthread_mutex_lock(&mutex); // Acquisizione del mutex

    // Verifica se il buffer e' vuoto
    while (count == 0) pthread_cond_wait(&empty, &mutex); // Attesa sulla variabile
    condizione empty

    // Legge l'elemento dal buffer
    item = buffer[count - 1];
    count--;

    leggi_buffer('C');

    pthread_cond_signal(&full); // Sveglia un produttore in attesa sulla variabile
    condizione full
    pthread_mutex_unlock(&mutex); // Rilascio del mutex

    return NULL;
}

int main(int argc, char* argv[]) {
    if (argc != 3) {
        printf("Usage: %s <np> <nc>\n", argv[0]);
        return 1;
    }

    int np = atoi(argv[1]); // Numero di produttori
    int nc = atoi(argv[2]); // Numero di consumatori
    N=np; // Dimensione del buffer

    buffer = malloc(N*sizeof(char)); // Allocazione del buffer
    pthread_t* producers = malloc(np*sizeof(pthread_t));
    pthread_t* consumers = malloc(nc*sizeof(pthread_t));

    // Creazione dei produttori
    for (int i = 0; i < np; i++) pthread_create(&producers[i], NULL, producer, NULL);

    // Creazione dei consumatori
    for (int i = 0; i < nc; i++) pthread_create(&consumers[i], NULL, consumer, NULL);

    // Join dei produttori
    for (int i = 0; i < np; i++) pthread_join(producers[i], NULL);
}

```

```
// Join dei consumatori
for (int i = 0; i < nc; i++) pthread_join(consumers[i], NULL);

free(producers);
free(consumers);

return 0;
}
```